Lab -1

Analysis and Structural Design Tools.

System analysis and design tools give analyst a structured way to ~~std~~ study an existing or proposed system before it is actually built. Instead of approaching a system as one large, tangled unit, these tools break it down into smaller, more manageable pieces, which makes the overall design easier to plan, document and communicate to others.

The 3 commonly used tools are:

- Data Flow ~~Dig~~ Diagram (DFD)
- Entity Relationship Diagram (ERD)
- Class diagram (Object-Oriented system structure)

A: Data Flow Diagram (DFD)

A Data Flow Diagram is a visual modeling technique used during system analysis to trace the path that data follows as it travels through a system. It shows how raw input is converted into meaningful output as it passes through a series of processing steps.

Notation used in DFD:

DFD rely on small set of standardized shapes each representing a different system element:

Process:

Represents an activity or function that converts incoming data into outgoing data.

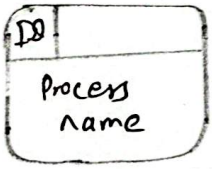


Fig: Process block.

Data Flow:

Represents data moving between a process, a data store and and external entity.

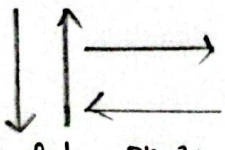
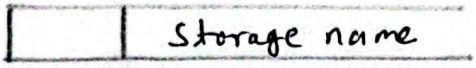


Fig: Data Flow.

Data Store:

Represents a location where data is held.



External Entity (source/sink)

Represents a person, organization, or system outside the boundary of the system.

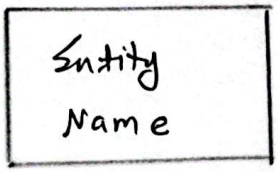


Fig: Source/sink.

*Level 0 (Context diagram)

Shows the entire system as a single process and depicts only its interaction with outside entities. This level gives a high-level, big-picture view without exposing internal workings.

*Level 1 DFD:

Expands the single process from level 0 into several sub-processes and exposes the major data flows that occur between

*Level 2 DFD:

Takes each sub-process from Level 1 & breaks it down further into more detailed steps. This decomposition continues until the diagram contains enough detail to be useful for implication.

A-1: Draw data flow diagram for Online Shopping system.

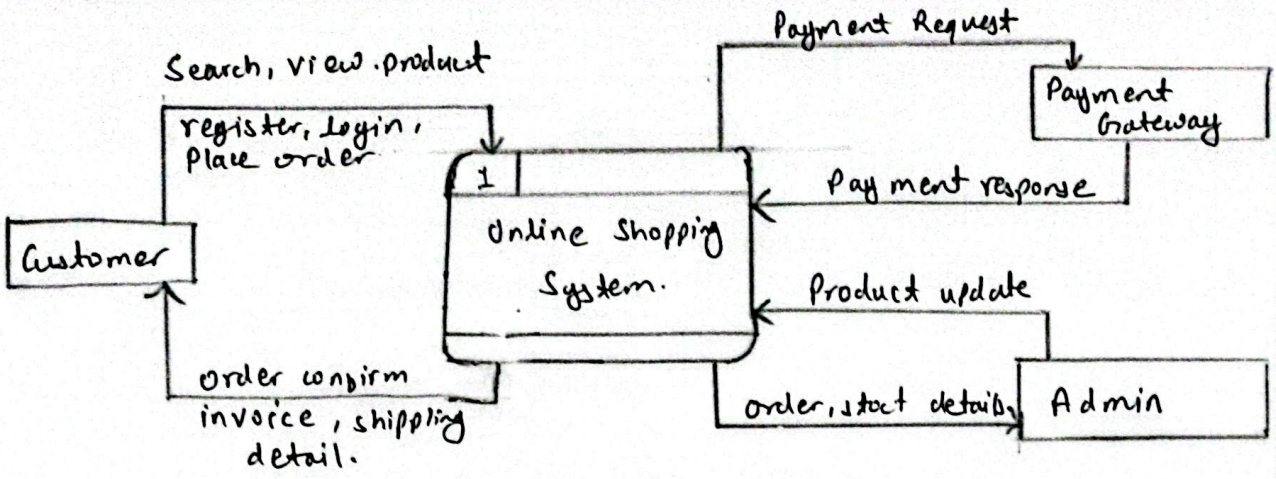


Fig: Context Diagram for Online Shopping:

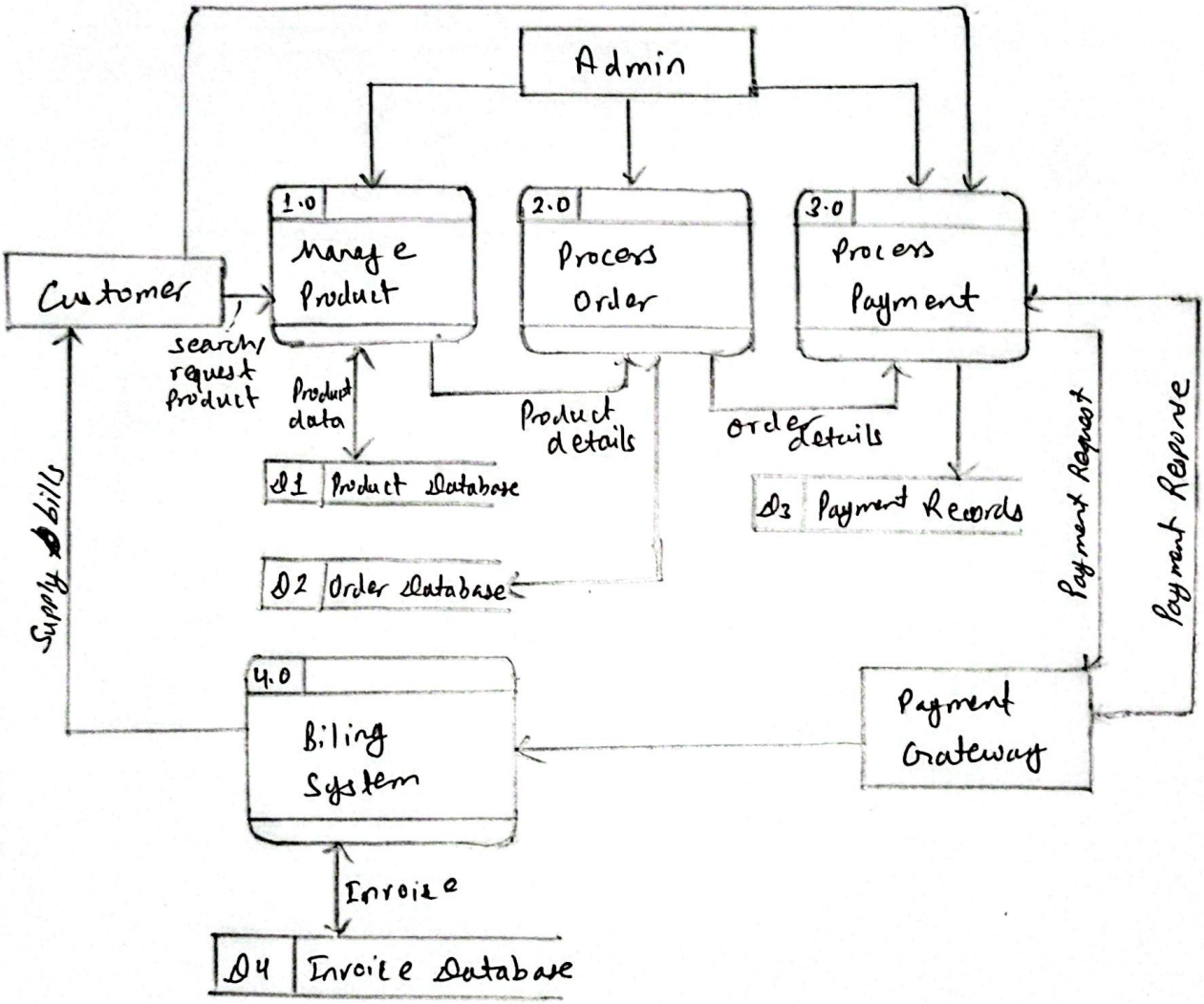


Fig: DFD for Online Shopping System

A:2: Design a DFD for Student ~~Management~~ Attendance Management System.

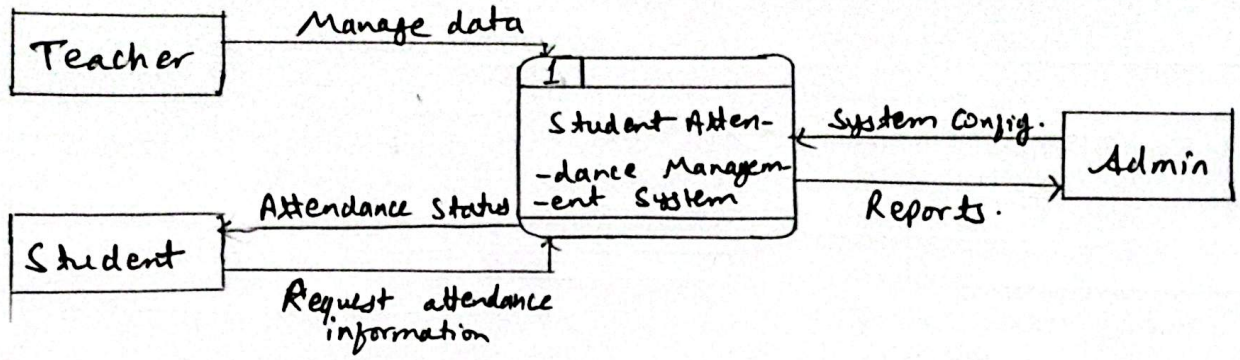


Fig: Context diagram

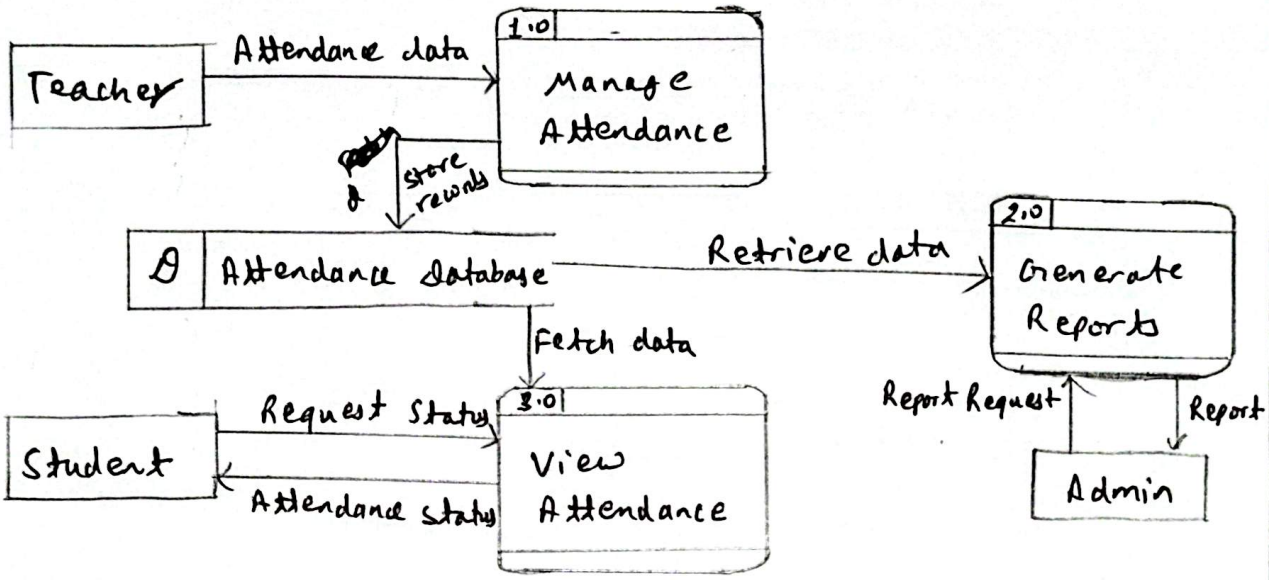


Fig: Level 1 DFD for Student Student Account Management system.

A.3: DFD for Library Management System:

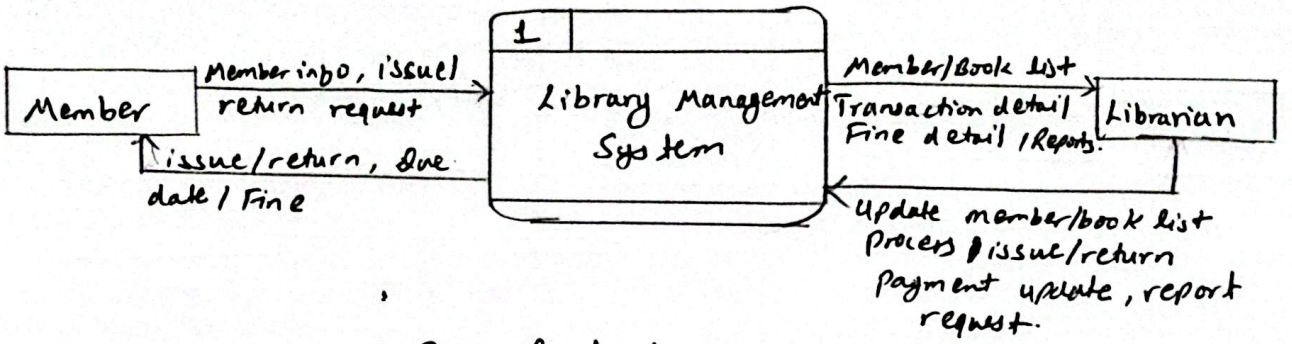


Fig: Context diagram.

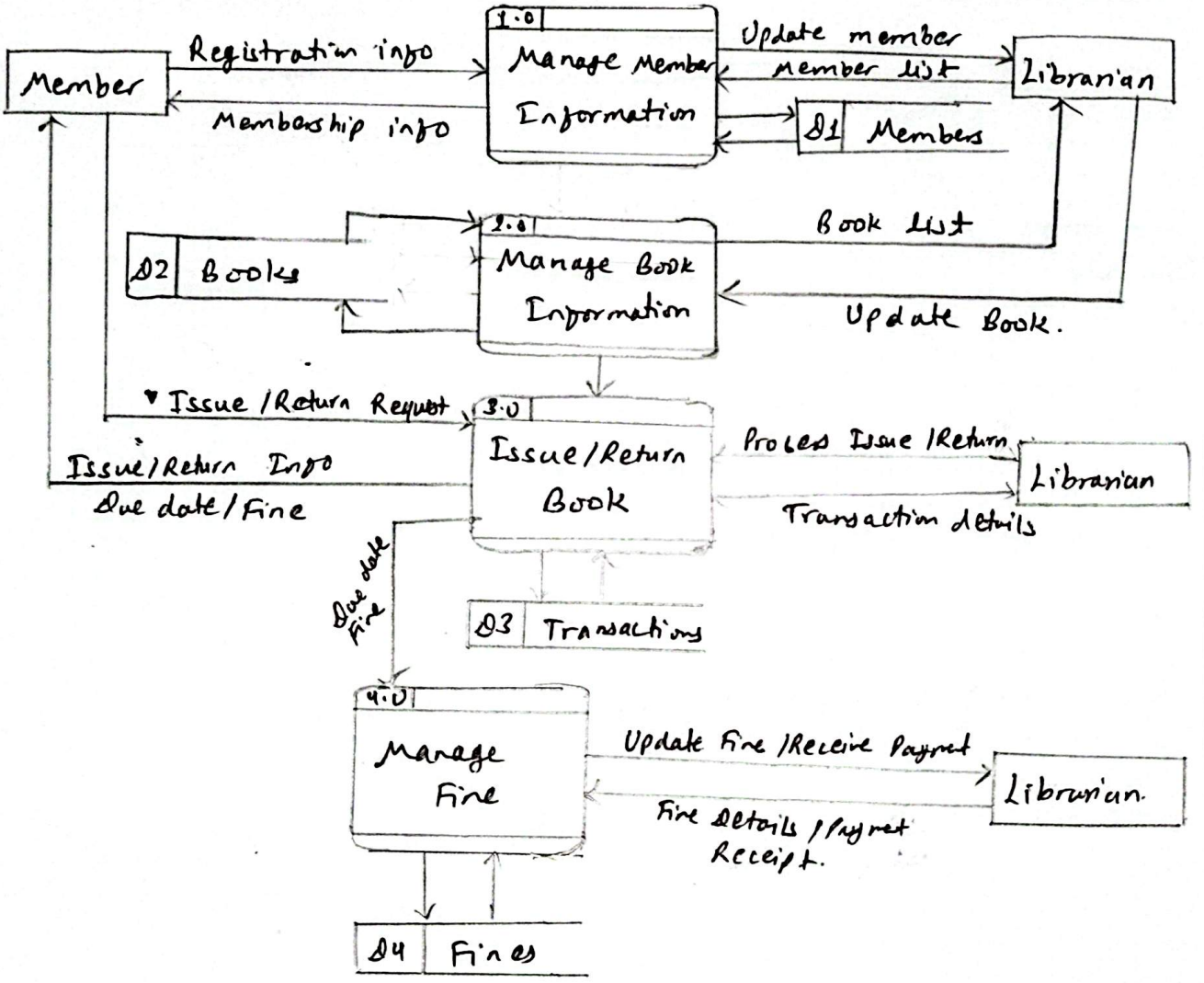


Fig: DFD for Library Management System.

A.4 : DFD for Hotel front office System:

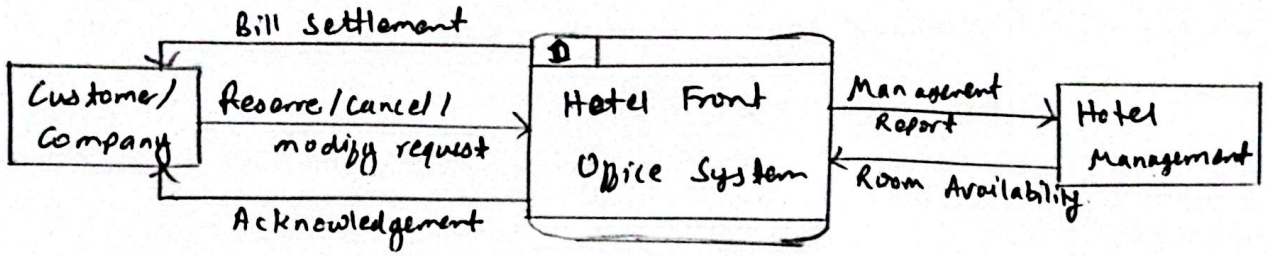


Fig: Context diagram.

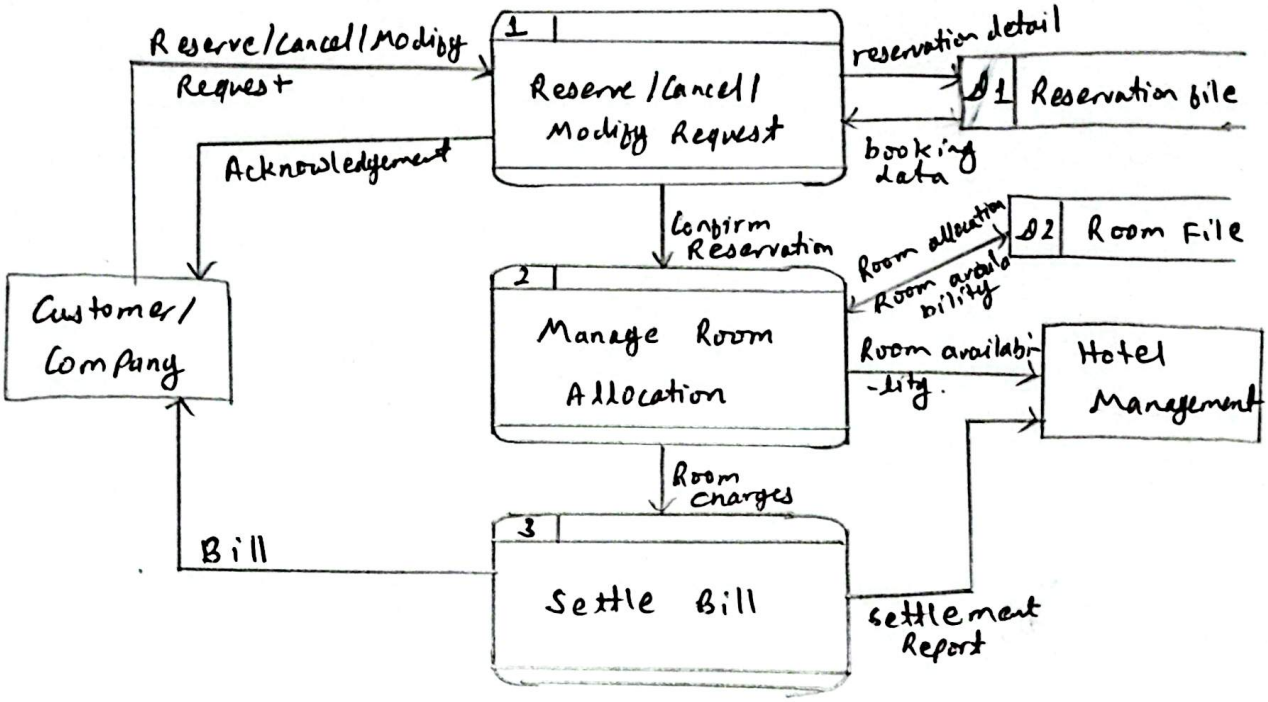


Fig: DFD for Hotel front office system

A.5: Draw DFD for Student Information system upto level 2.

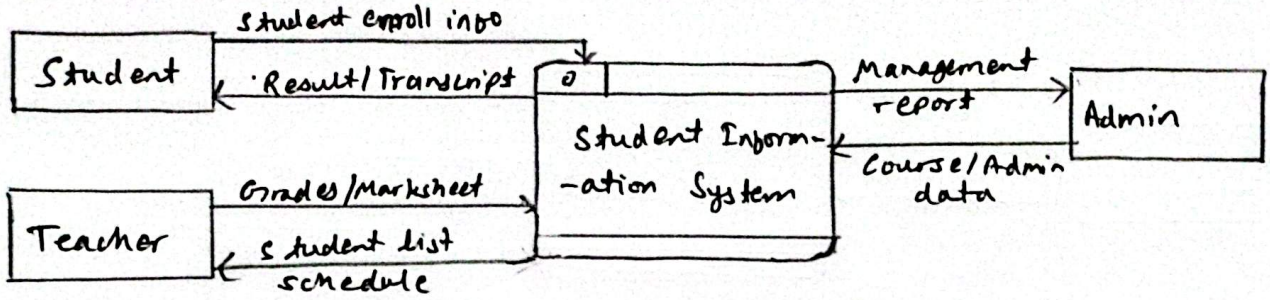


Fig: Context diagram

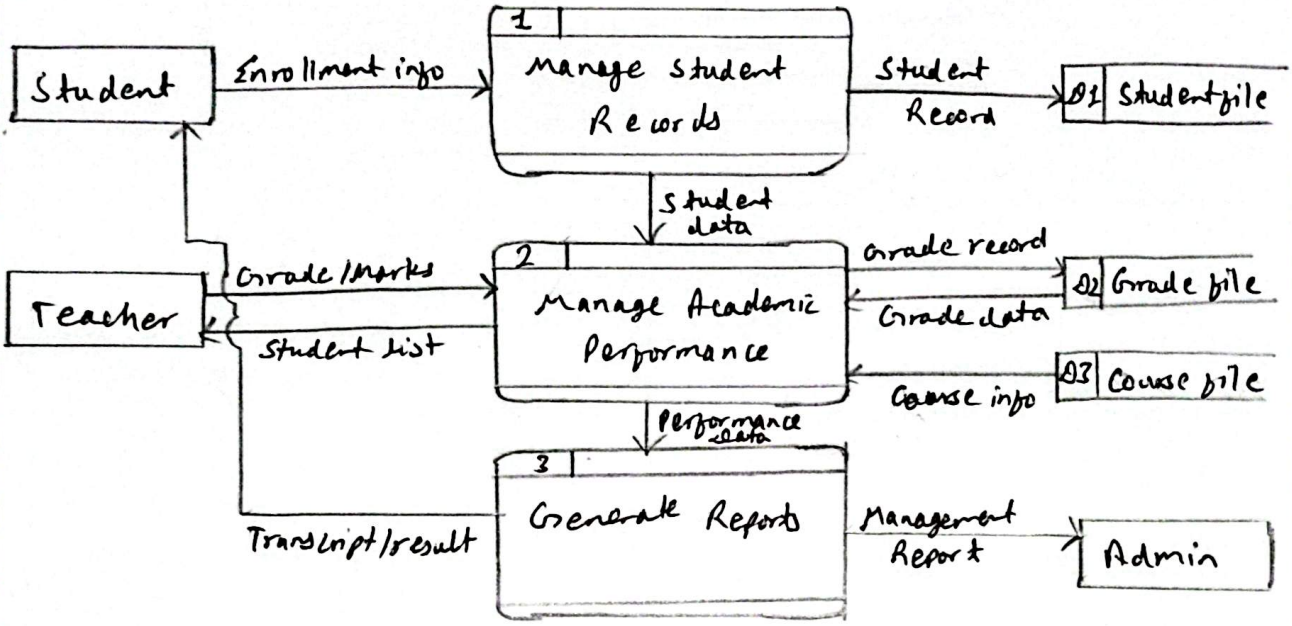


Fig: Level 1 DFD

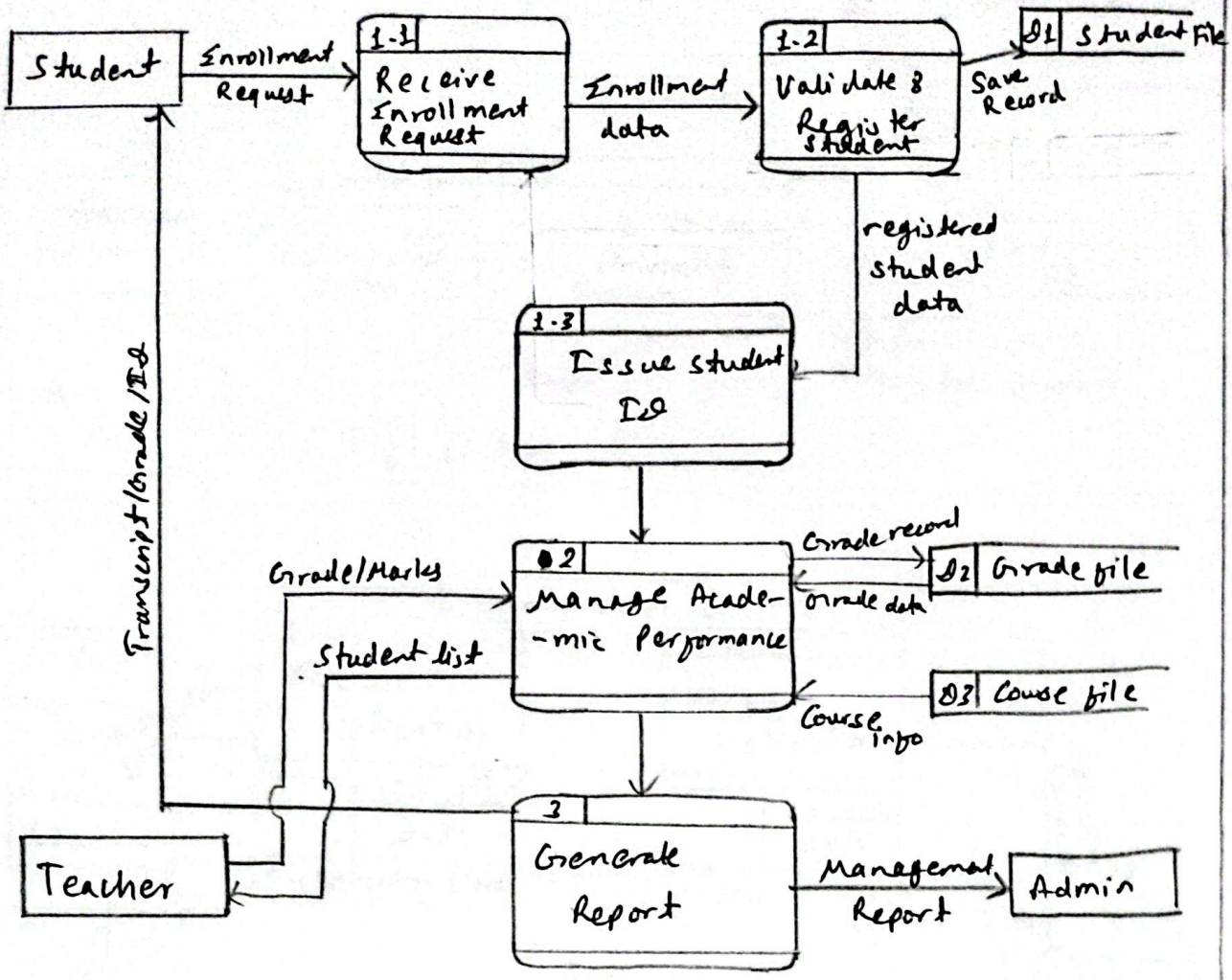


Fig: Level 2 DFD

A:6: Draw context diagram and DFD for distance education university:

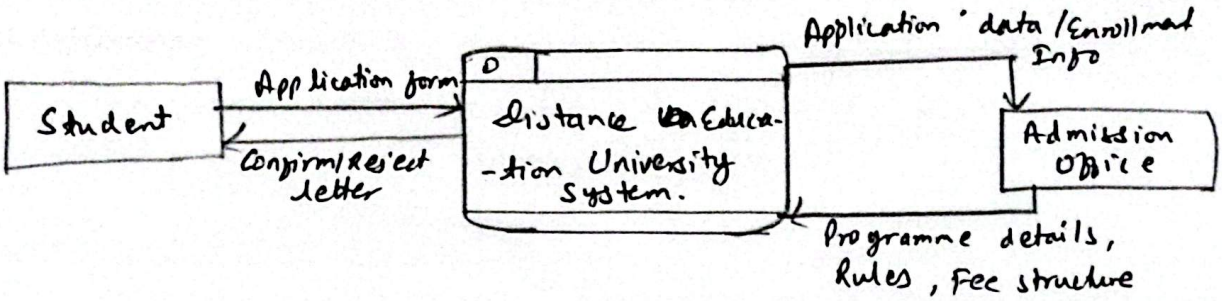


Fig: Context diagram

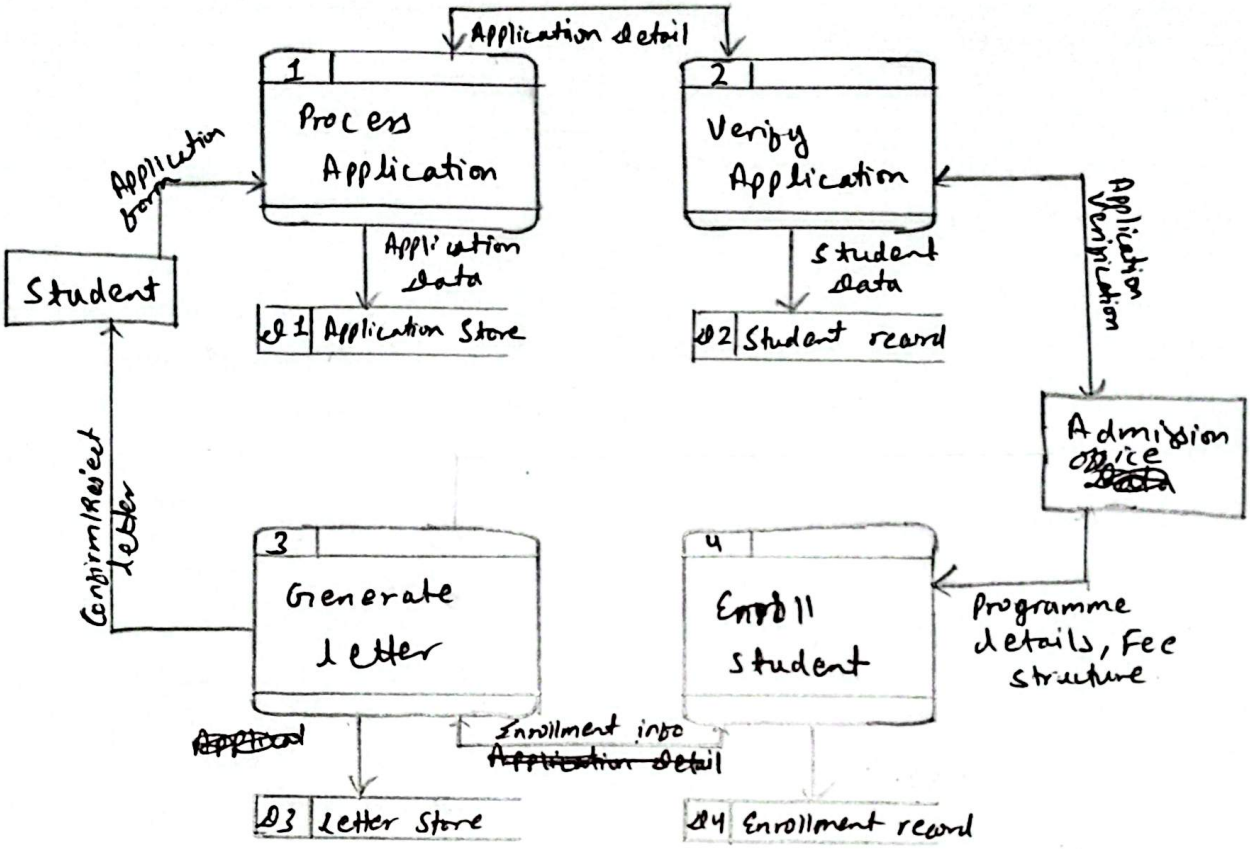
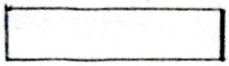
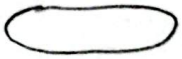
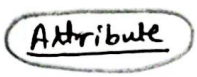


Fig: DFD for distance Education University:

B: Entity-Relationship Diagram (ERD)

An ERD is a diagrammatic technique used in database design to capture entities, the attributes that describe them, and the relationships connecting them to one another. It helps designers to organize data in logical structure that can later be translated into actual database tables.

Symbols used in ERD:

S.N	Symbol Name	Shape	Description.
1.	Entity	 Rectangle	- Represents real world object or concept.
2.	Attribute	 Oval	- A property or characteristic of an entity
3.	Derived Attribute	 Dotted oval	- An attribute whose value can be calculated from other attributes
4.	Multivalued Attribute	 Double oval	- An attribute capable of having more than one value
5.	Primary Key Attribute		- Uniquely identifies each record in relation
6.	Relationship	 Diamond	- The connection or association between two entities
7.	Weak-Strong Relationship	 Double diamond.	- Represent relation between weak & strong entity it depends on

B:1: Draw ER diagram for hospital management System.

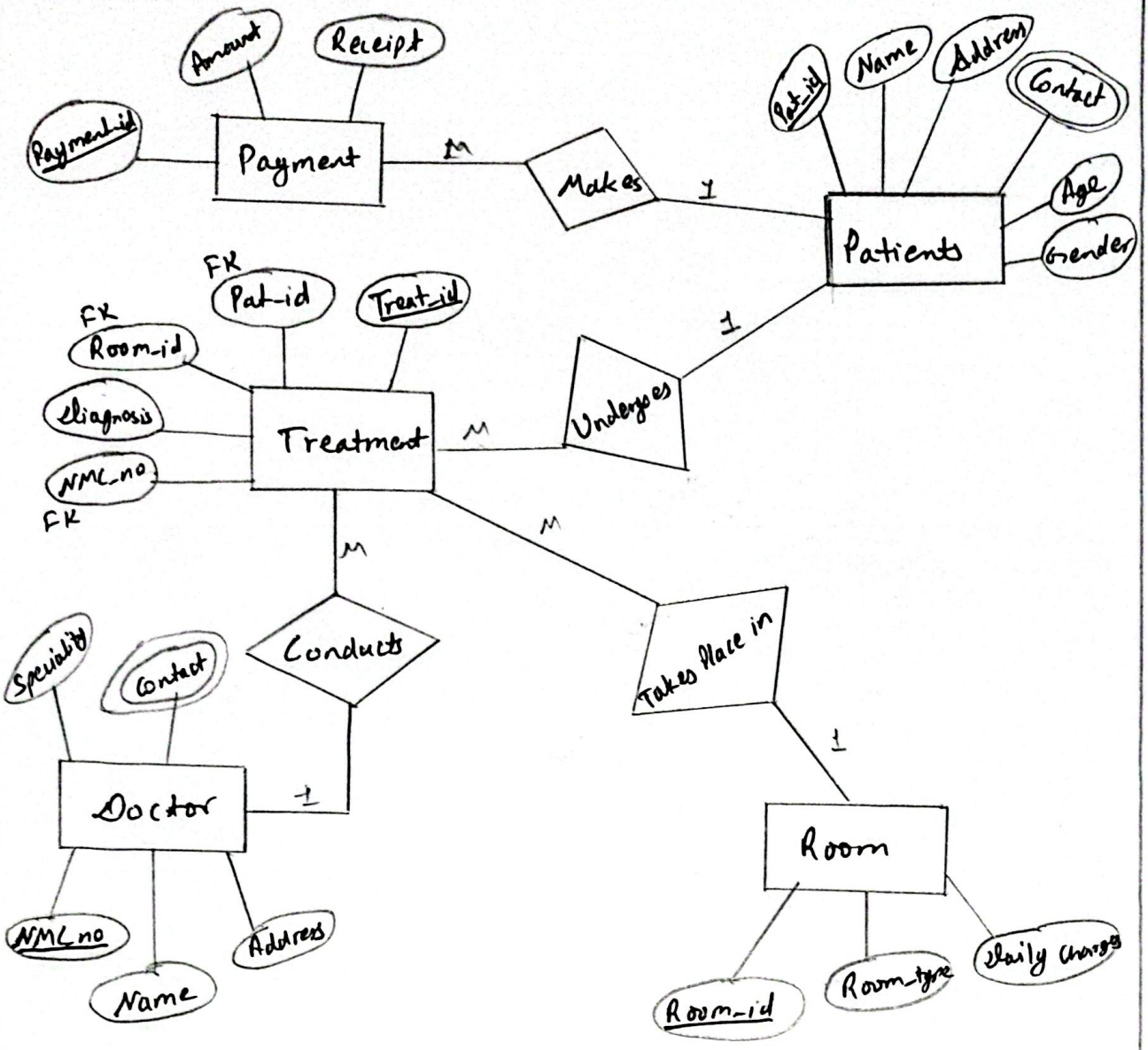


Fig: Er diagram of Hospital Management System.

B:2: Draw ER diagram for student Attendance Management System

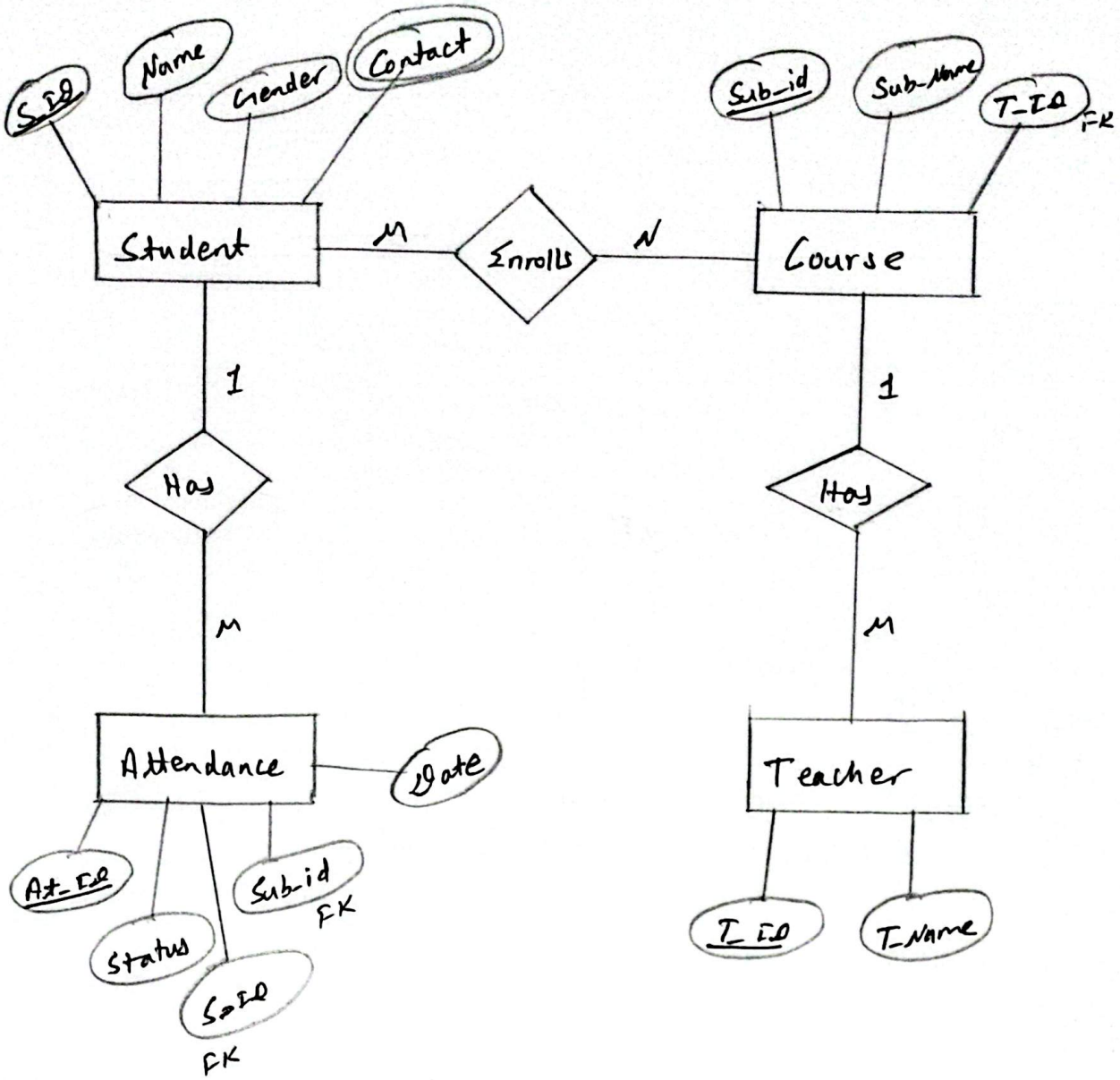


Fig: ER-diagram of Student Attendance Management System

B:3: Draw ER diagram for library management system..

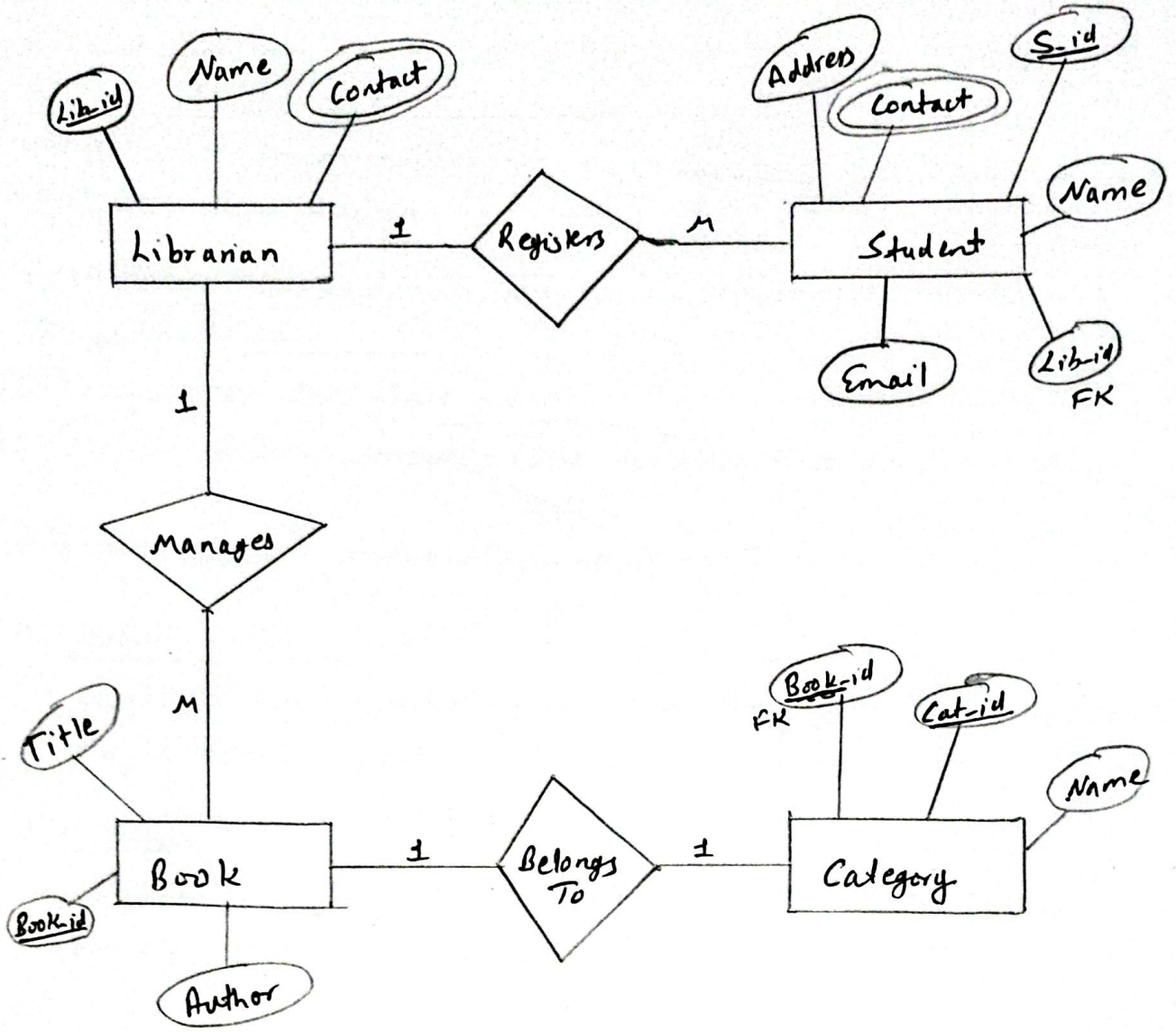


Fig: ER diagram for Library Management System

c) Class diagram:

A class diagram is a structural diagram from the Unified Modeling Language (UML) used to represent the static design of a system. It lays out the classes that make up a system along with their attributes, their central to object-oriented analysis and design.

Each class is ~~divided~~ drawn as a rectangle divided into 3 compartments:

(1) Top section: the class name.

(2) Middle section: attributes (the variables that hold the class's data).

(3) Bottom section: methods/operations

* Attributes:

- Captures the properties of a class. For e.g. a 'person' class might have attributes such as 'name', 'age'.

* Methods:

- Capture the functions or behaviors a class can perform. For e.g. display(), calculate().

In short, class diagram works as the blueprint of a system: by laying out classes and the relationships among them, it helps developers understand, design, and implement an object oriented system effectively.

C1: Design class diagram for Online Food Ordering System.

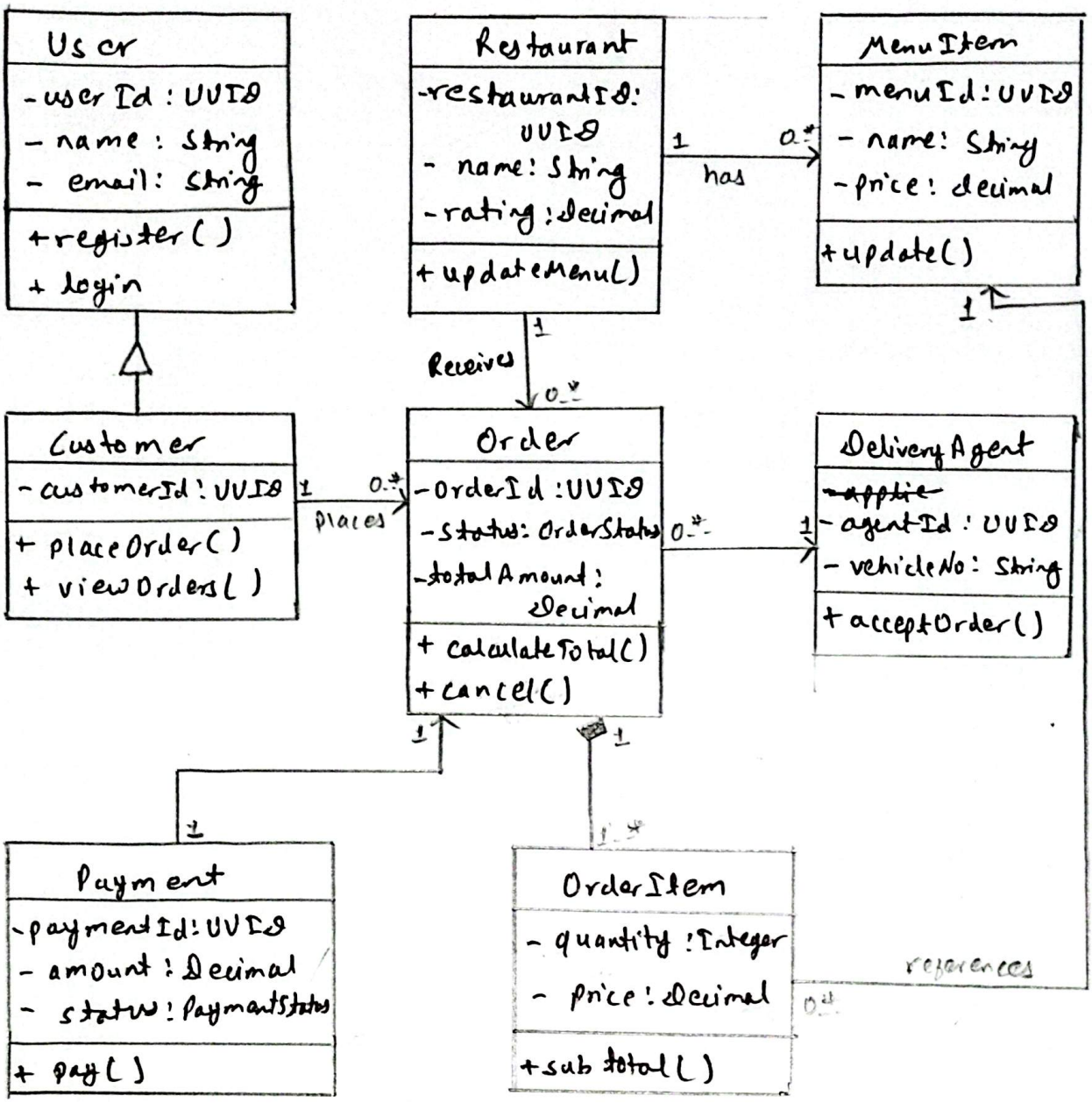


Fig: Class diagram for Online Food Ordering System

L2: Class Diagram for Library Management System.

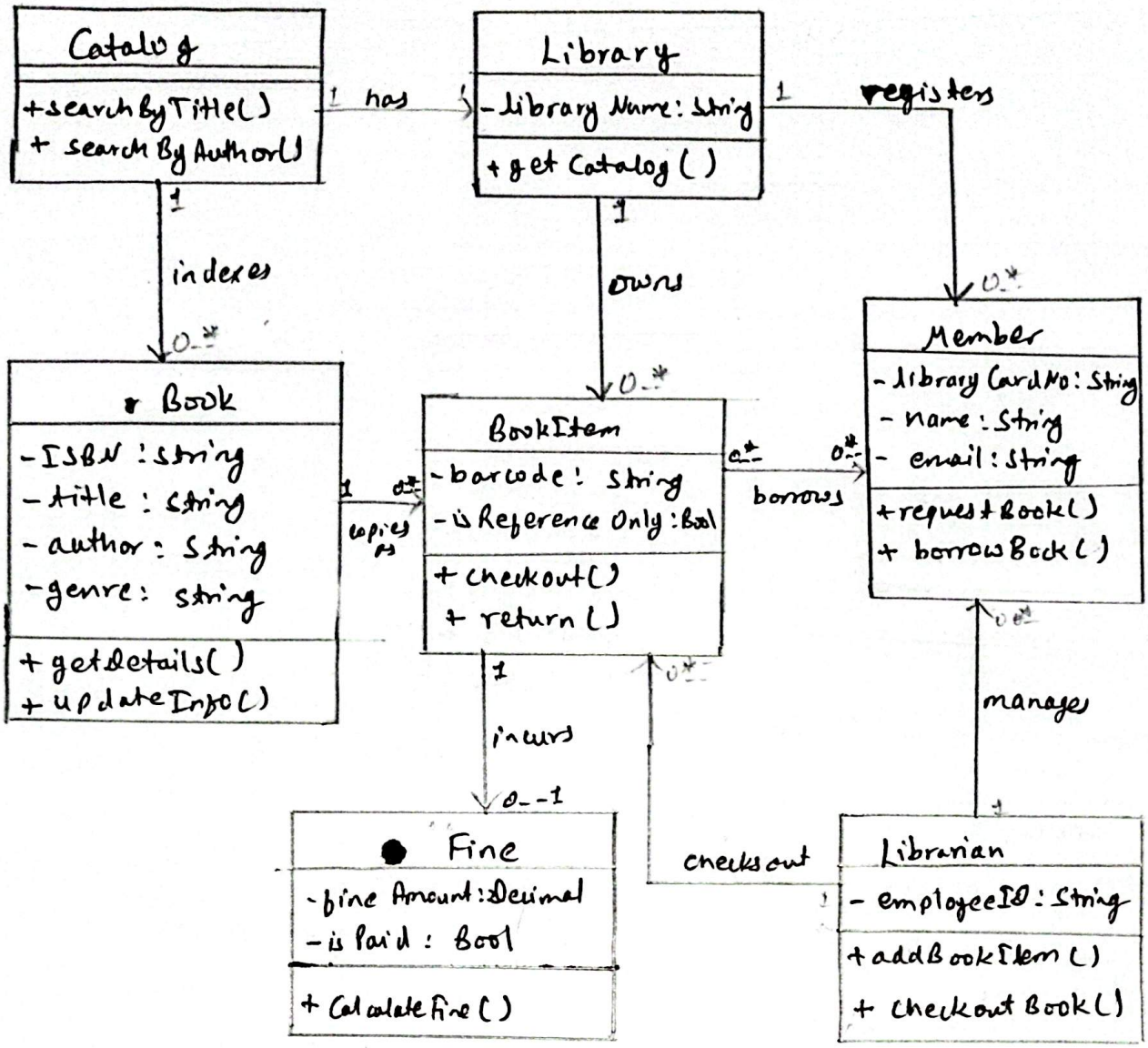


Fig: Class Diagram for Library Management System.

C3: Class diagram for Student Information Management System.

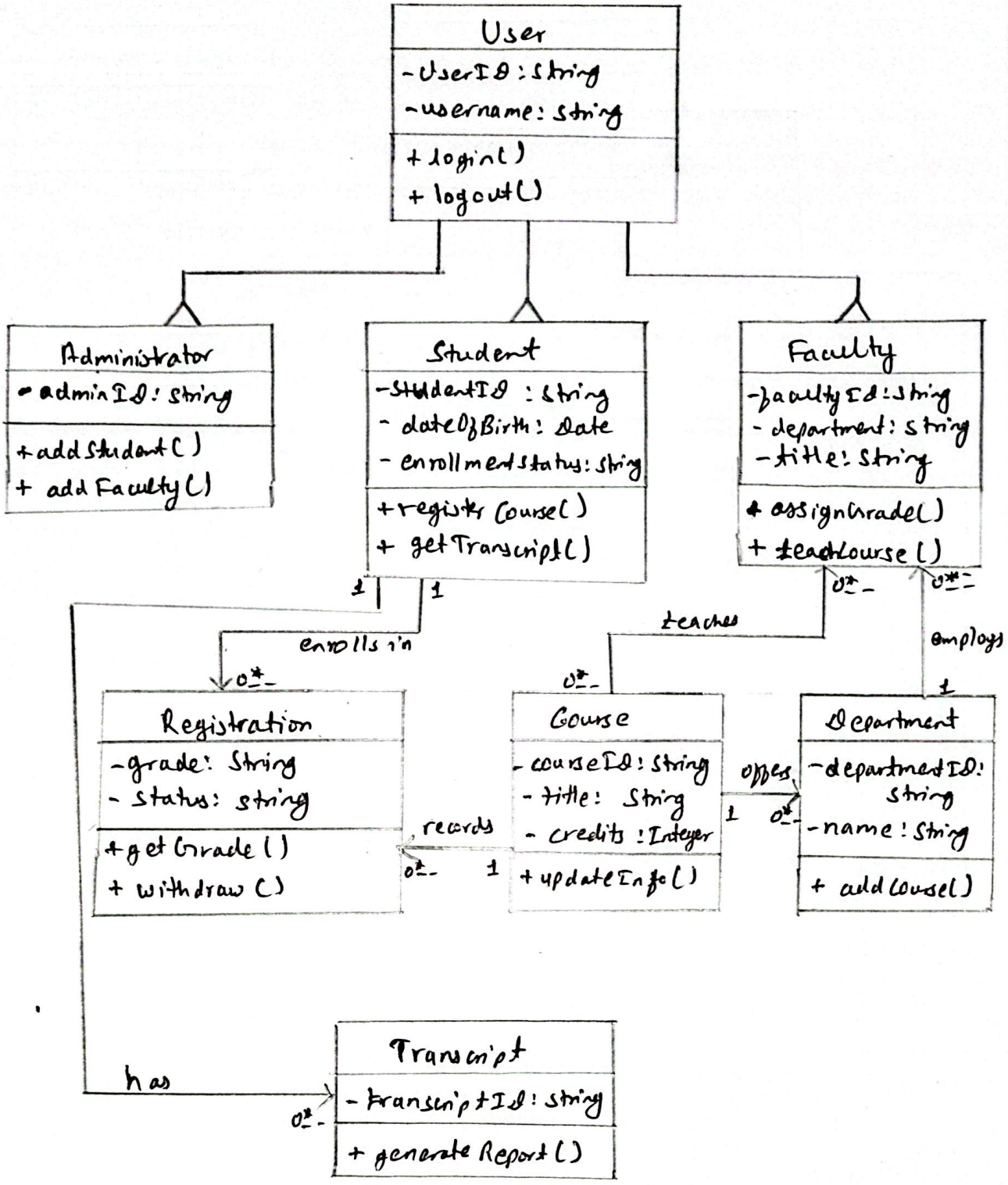


Fig: Class diagram for Student Information Management System

LAB-2

Behavioral / Dynamic Tool:

Behavioral tools capture how a system behaves over time: the timing of events, the changes the system goes through, and the way its different parts interact. These tools focus on outlining the requirements and the communication flow between the user and the system, without necessarily fixing the exact sequence in which events occur.

A) Activity Diagram:

An activity diagram models the workflow or business process of a system. It behaves much like an enhanced flowchart, illustrating both the step-by-step sequence of activities and any activities that occur in parallel.

Notation used in an Activity Diagram:

 → Start node: mark the workflow begin

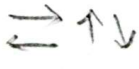
 → End node: mark the ending of workflow

 → represent one specific task or action (Action node/state).

 → Decision node: shows where the flow branches depending upon condition

 Fork: splits a single flow into multiple activities running in parallel.

 Join: brings parallel activities back together into one flow

 Control flow: an arrow that guides or indicates the direction of process.

A-1: Activity diagram for Job Portal System

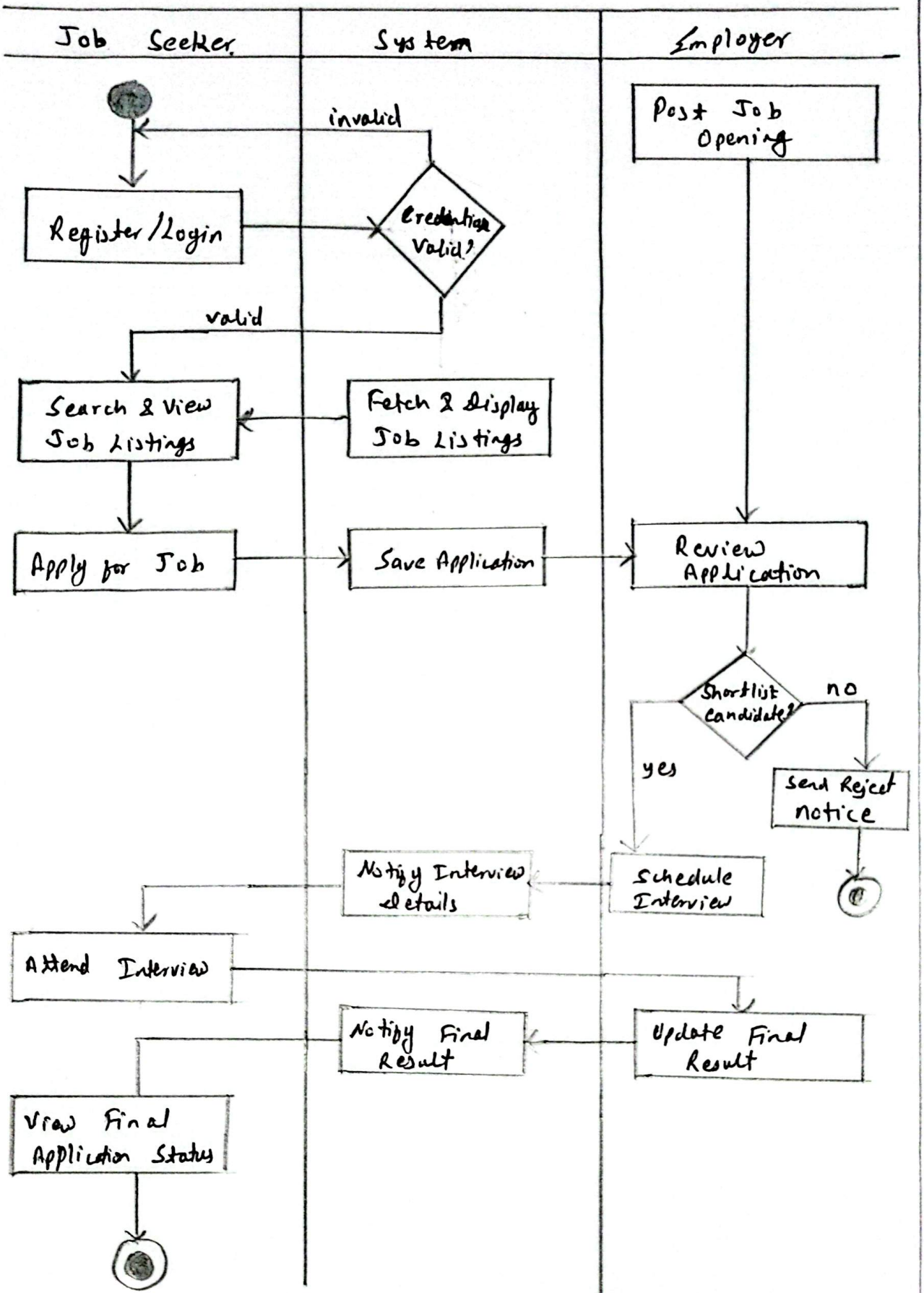


Fig: Activity diagram for Job Portal System

A:2: Activity diagram for Online Shopping:

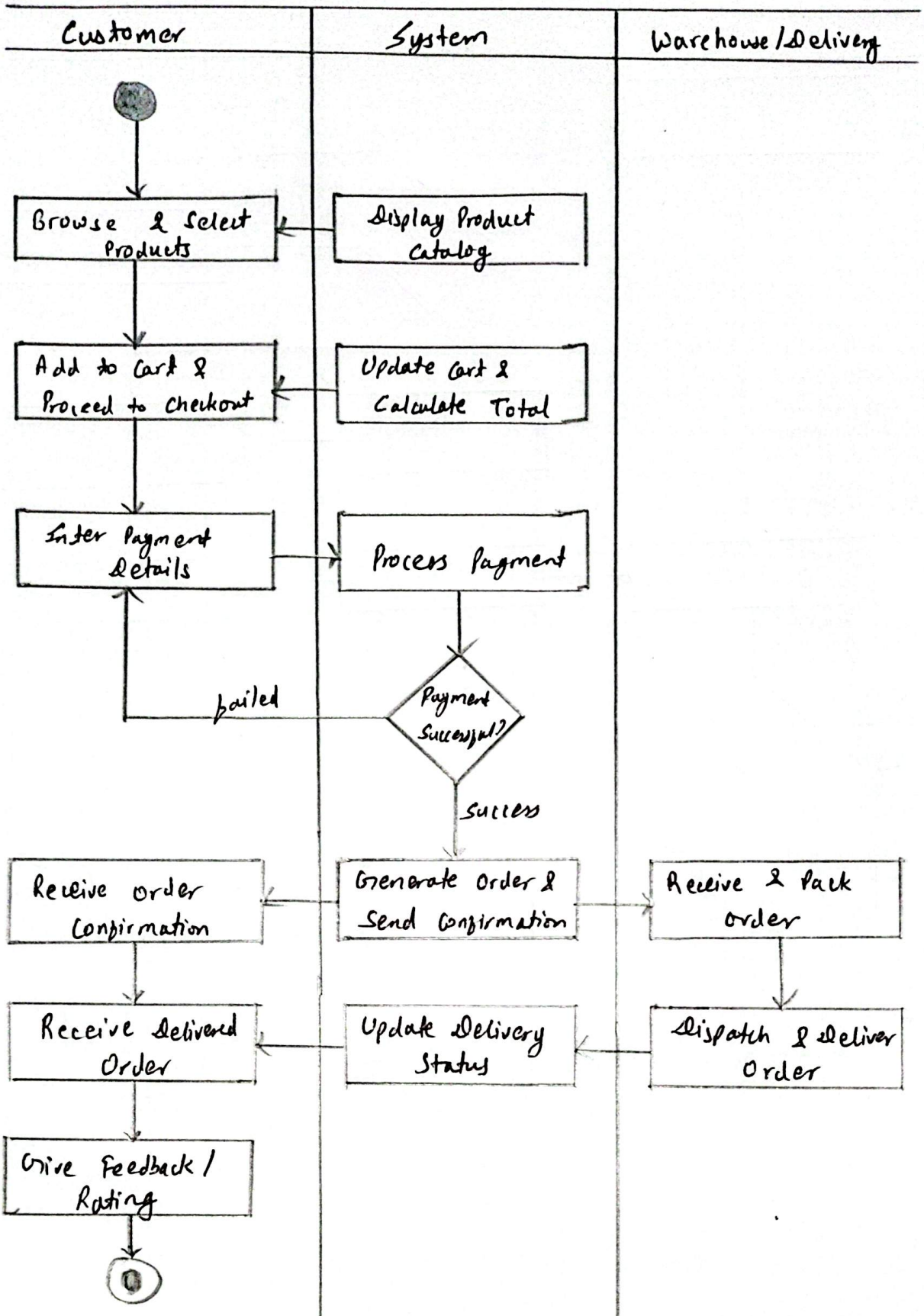


Fig: Activity Diagram for Online Shopping System

A:3: Activity diagram for Library Management System.

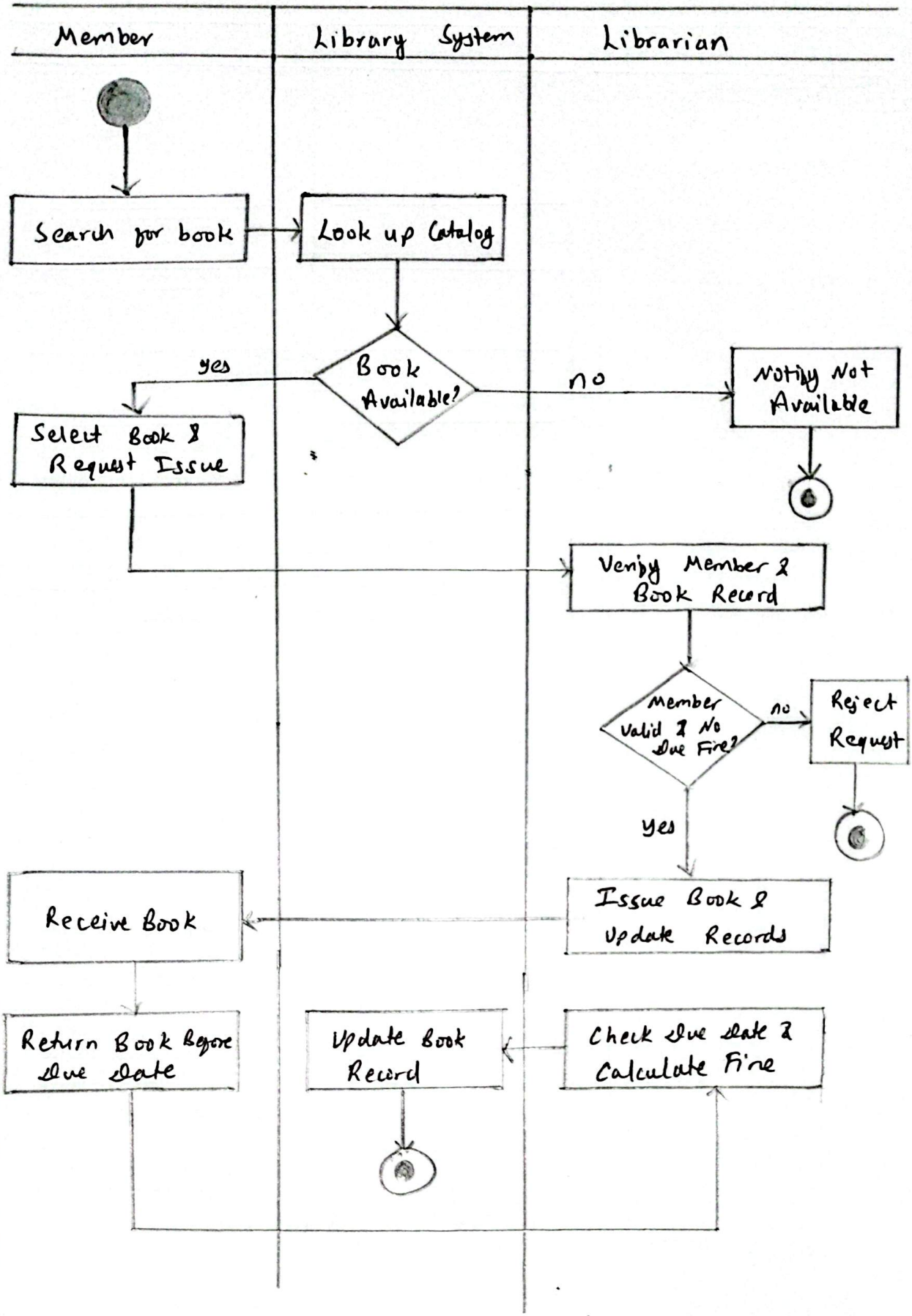


Fig: Activity Diagram for Library Management System.

A: 4' Activity diagram for Online Ticket Booking Process.

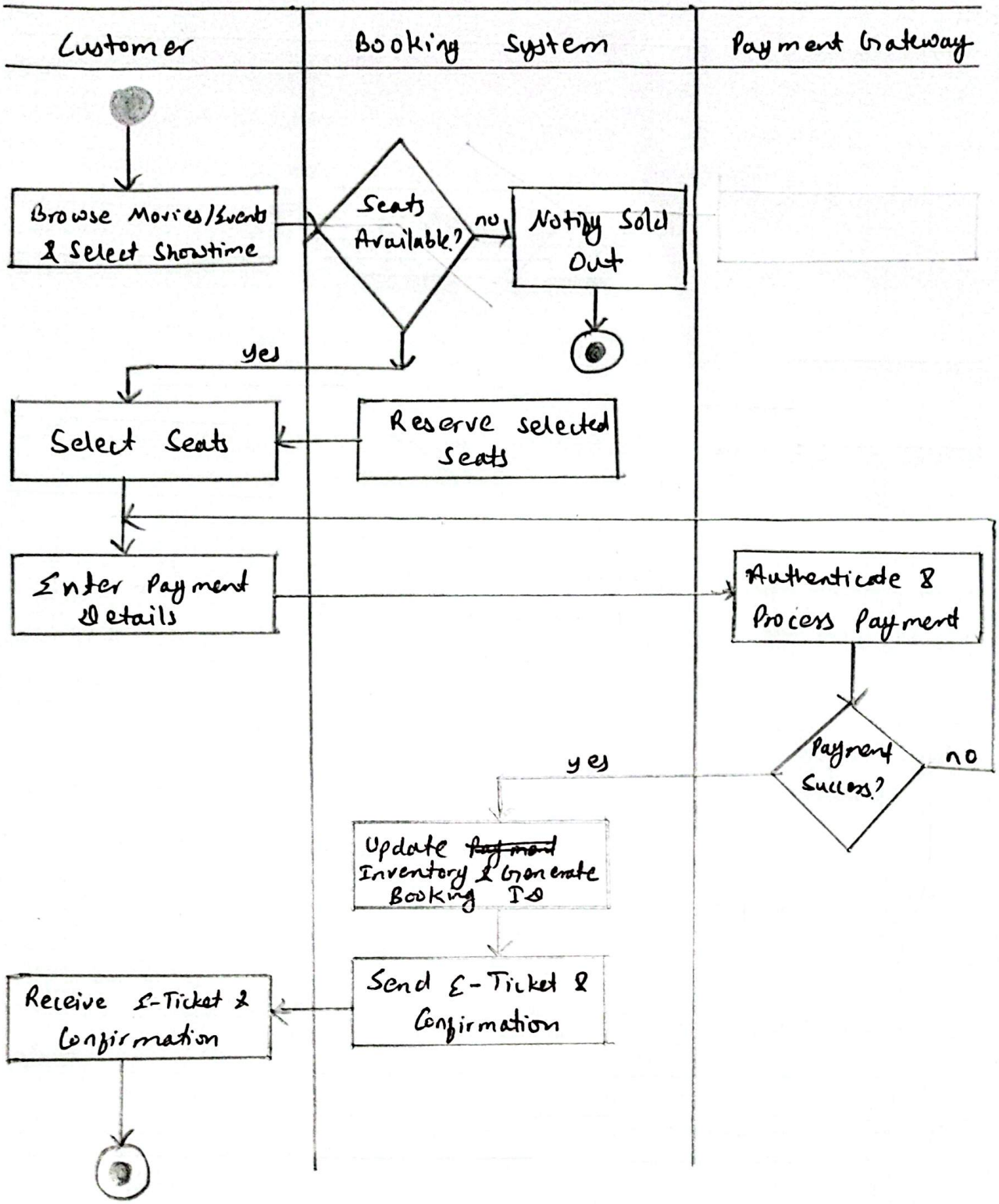
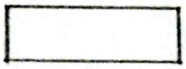


Fig: Activity diagram for Online Ticket Booking

B) Sequence diagram:

A sequence diagram is a UML behavioral diagram that shows how objects in a system interact with one another, arranged in the order in which those interactions occur. It is the most widely used dynamic modeling tool, & it works by tracking the messages exchanged between participants as they work together to complete a task.

Notations used in a Sequence diagram:

1)  → Object: Represents a system component that takes part in interaction

2)  → Actor: Represents an entity outside the system (a person or another system) that interacts with it.

3)  → Execution Occurrence: Shows the time period during which an object is actively performing an operation.

4) ——▶ Synchronous Message: Shows that the sender ~~carries~~ ~~pauses~~ pauses and waits for a response before continuing

5) ——→ Asynchronous Message: Shows that the sender carries on working immediately after sending, ~~after~~ without waiting for reply.

6) <<create>> - - → Create Message: Shows a new object / message being created

7) ←- - <<destroy>> Destroy Message: Shows that a previously created obj / message is being terminated.

8) Self Message:  → a message that an object sends to itself, looping back onto its own lifetime.

B.1. Sequence diagram for ATM system (money withdrawal)

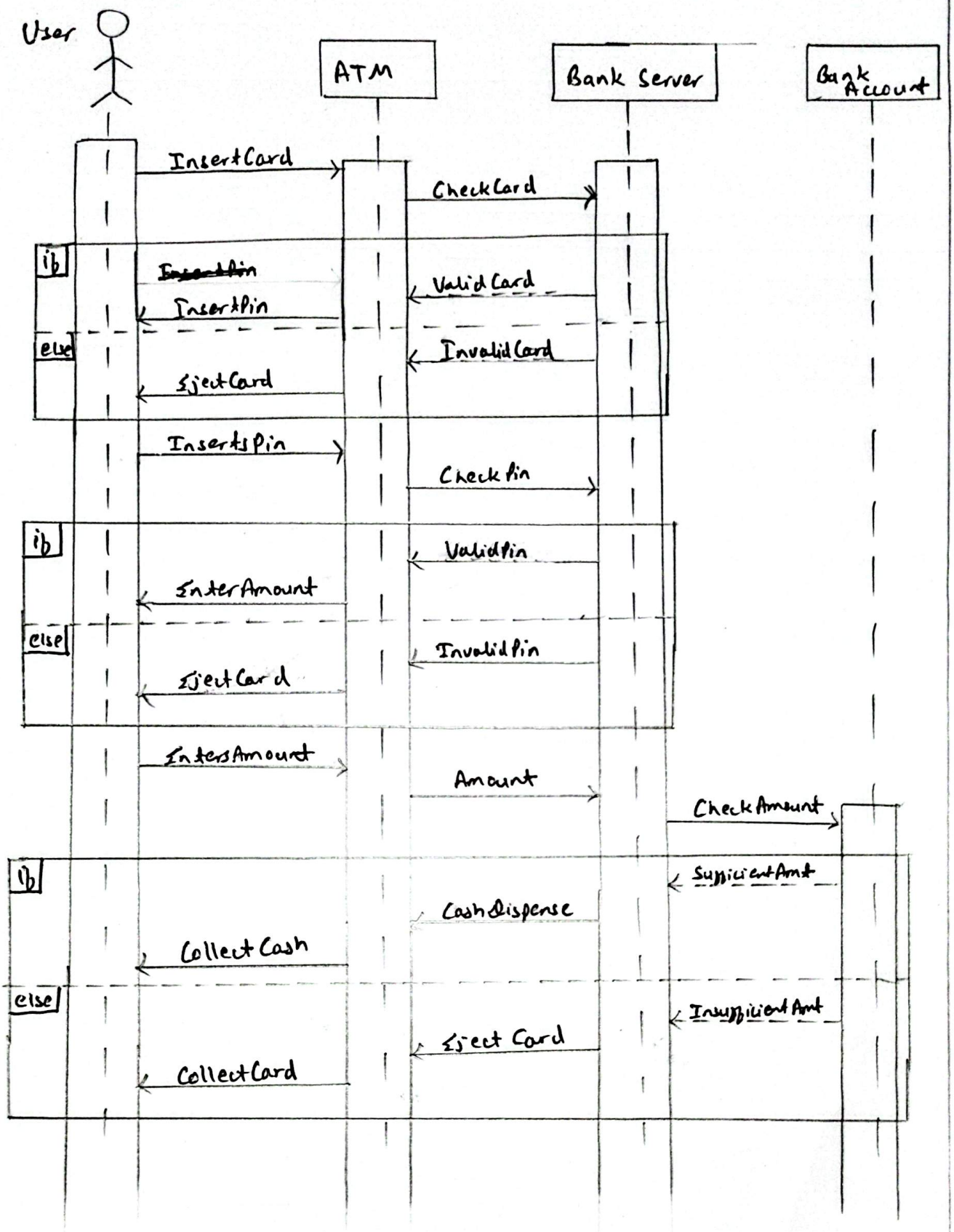


Fig: Sequence diagram For ATM system.

B:2. Sequence Diagram for Online Shopping.

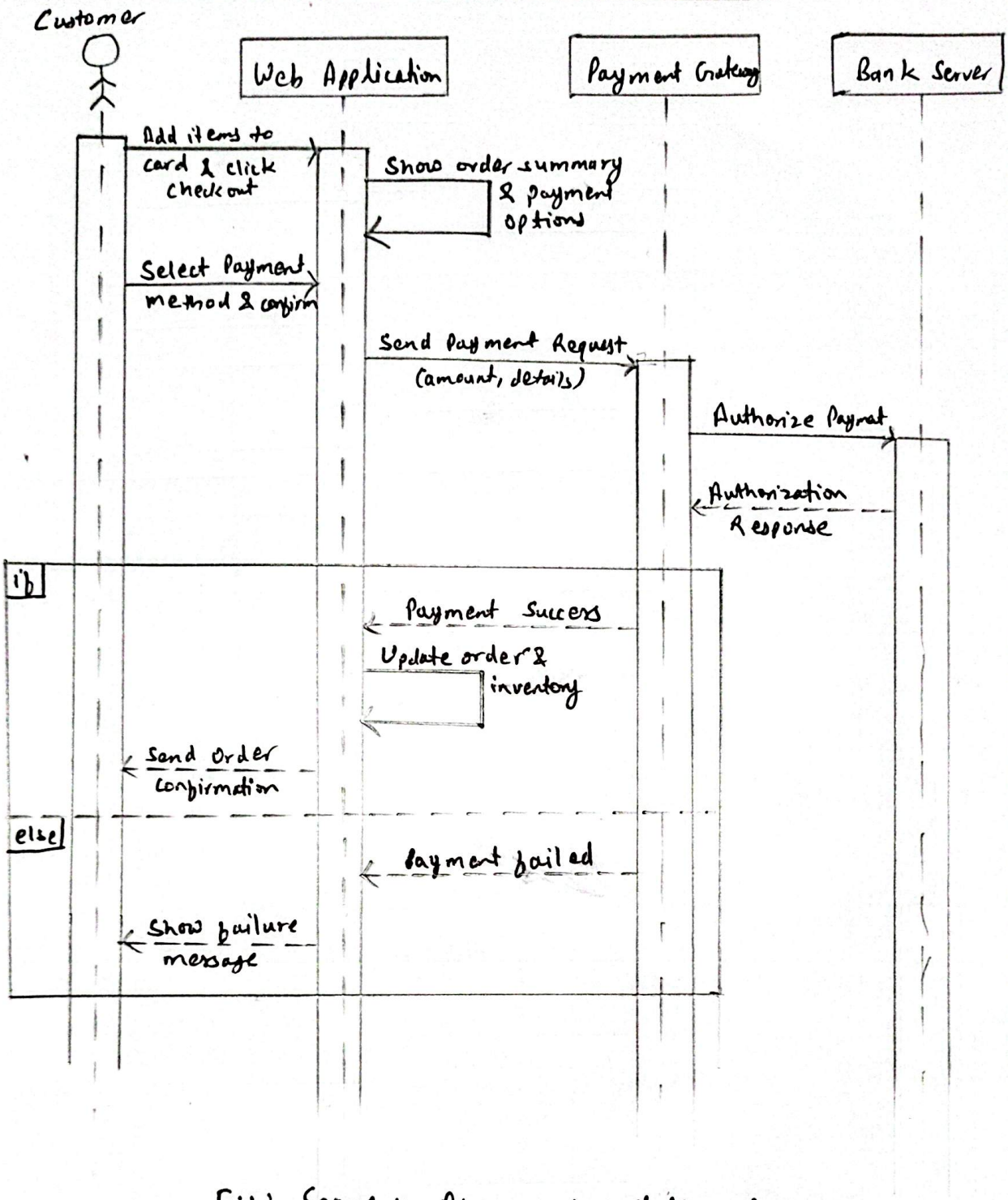


Fig: Sequence Diagram for Online Shopping

B3 : Sequence diagram for Borrowing Book from Library System

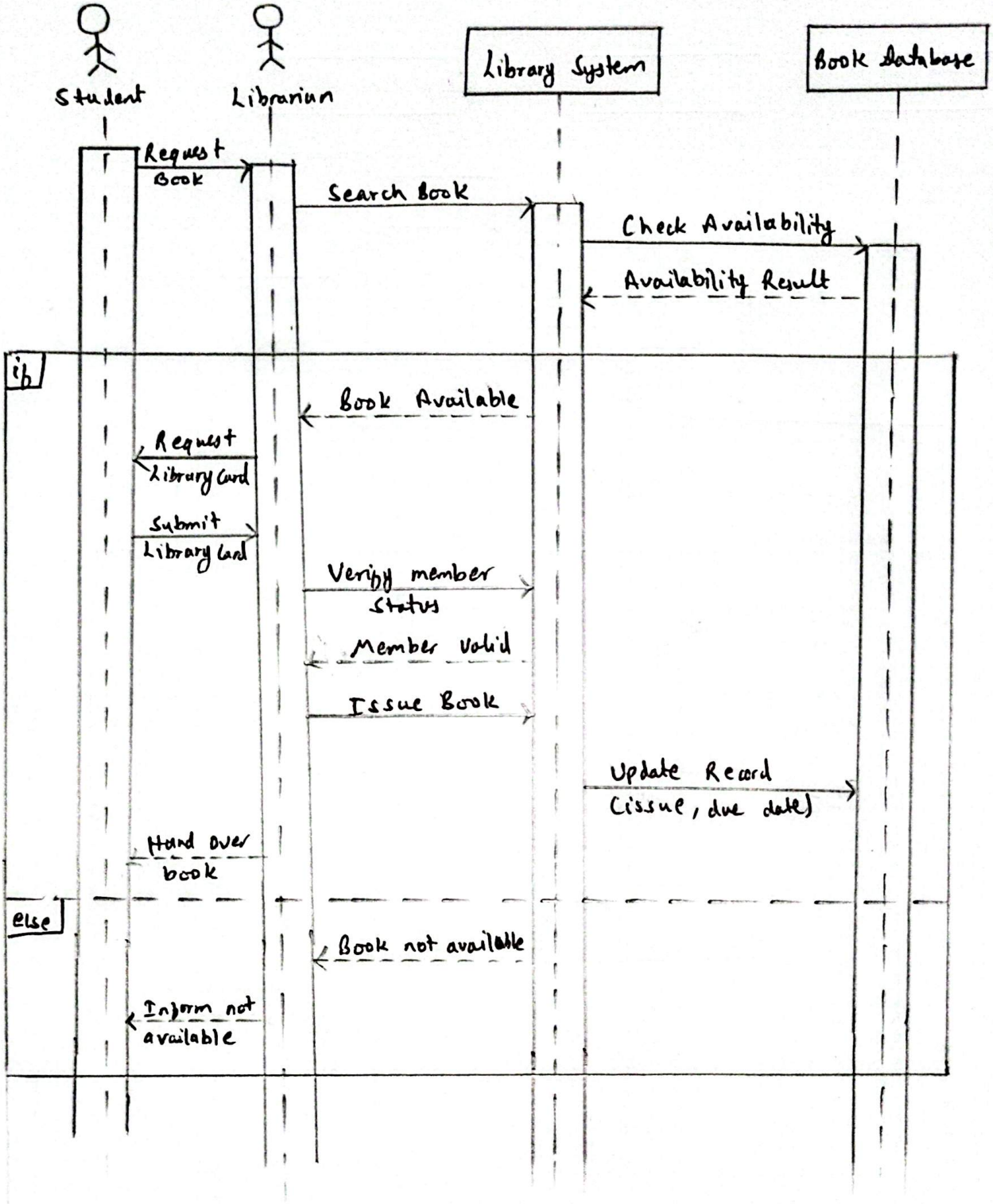


Fig: Sequence Diagram for Borrowing Book From Library System.

B.H: Sequence diagram for Online Payment System:

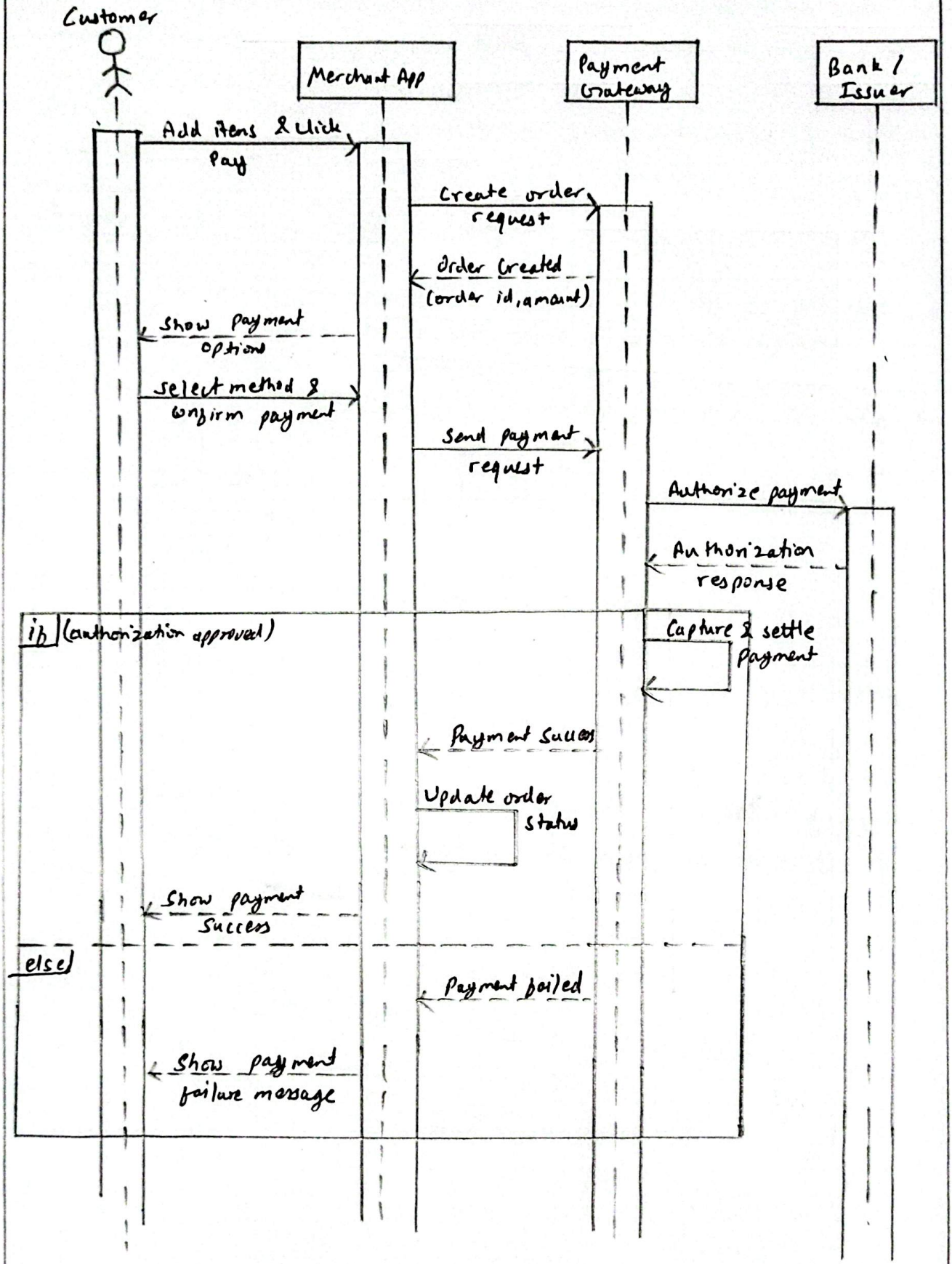
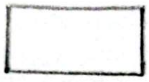


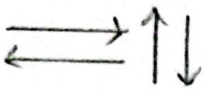
fig: Sequence diagram for Online Payment System

C) State Diagram:

A state diagram is another UML diagram, used to show how an object's condition changes over time as it reacts to events. It captures three things: the different states an object can occupy, the transitions that move it from one state to another, and the events that set off each transition.

Notation used in a State diagram:

1)  → State: represents a particular condition or situation an object is in at some point in its lifetime.

2)  → Transition: shows movement from one state to another

3)  → Initial state: Marks the starting point of the system/object

4)  → Final State: Marks the end of the process.

5) Events / Triggers: the actions or inputs that cause a transition to take place.

State diagrams are particularly suited to modeling systems whose behavior depends heavily on the order in which event occurs.

C.1: Design a state diagram for food delivery system.

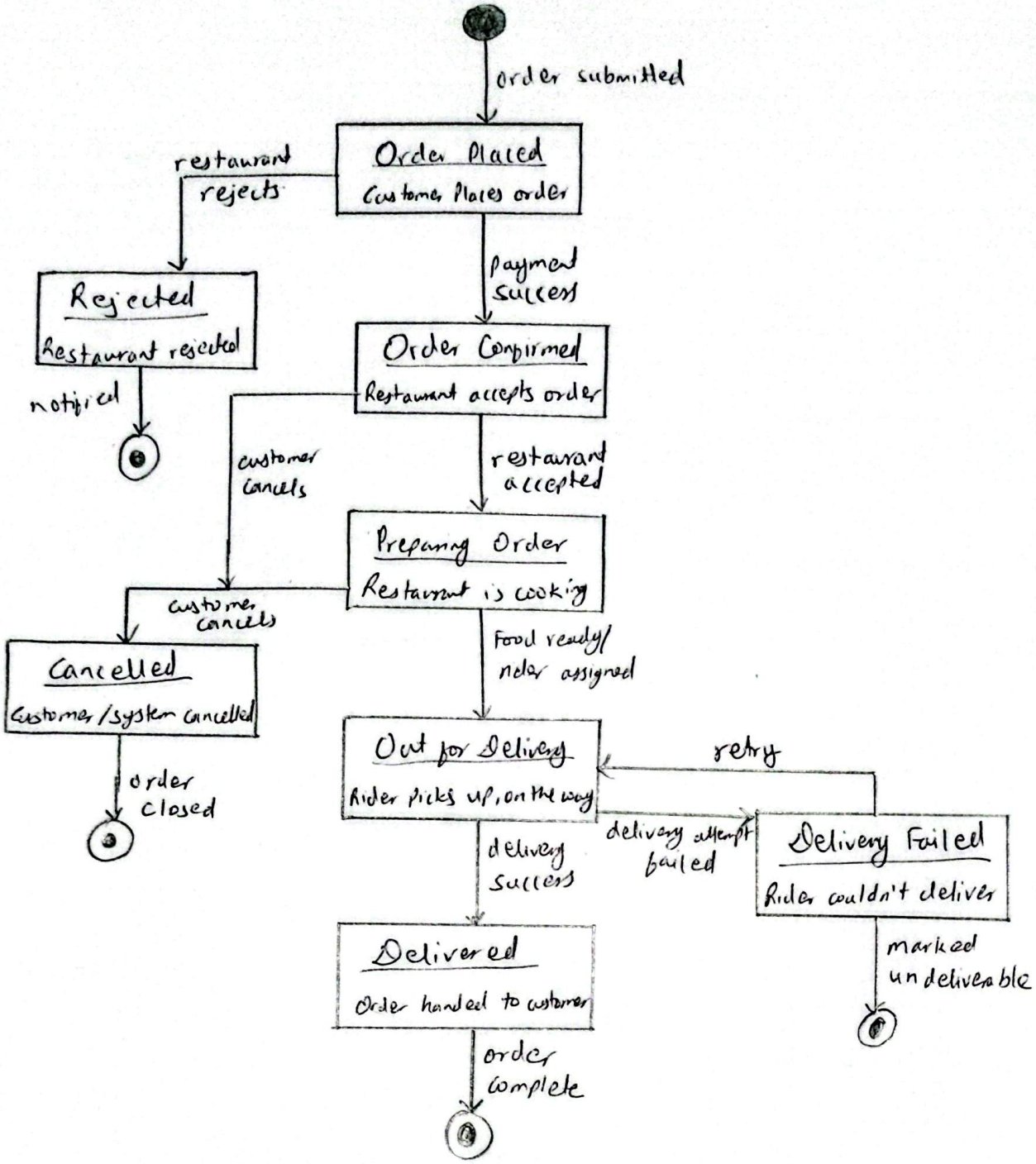


Fig: State Diagram for Food Delivery System.

C.2: Design the state diagram for online Payment

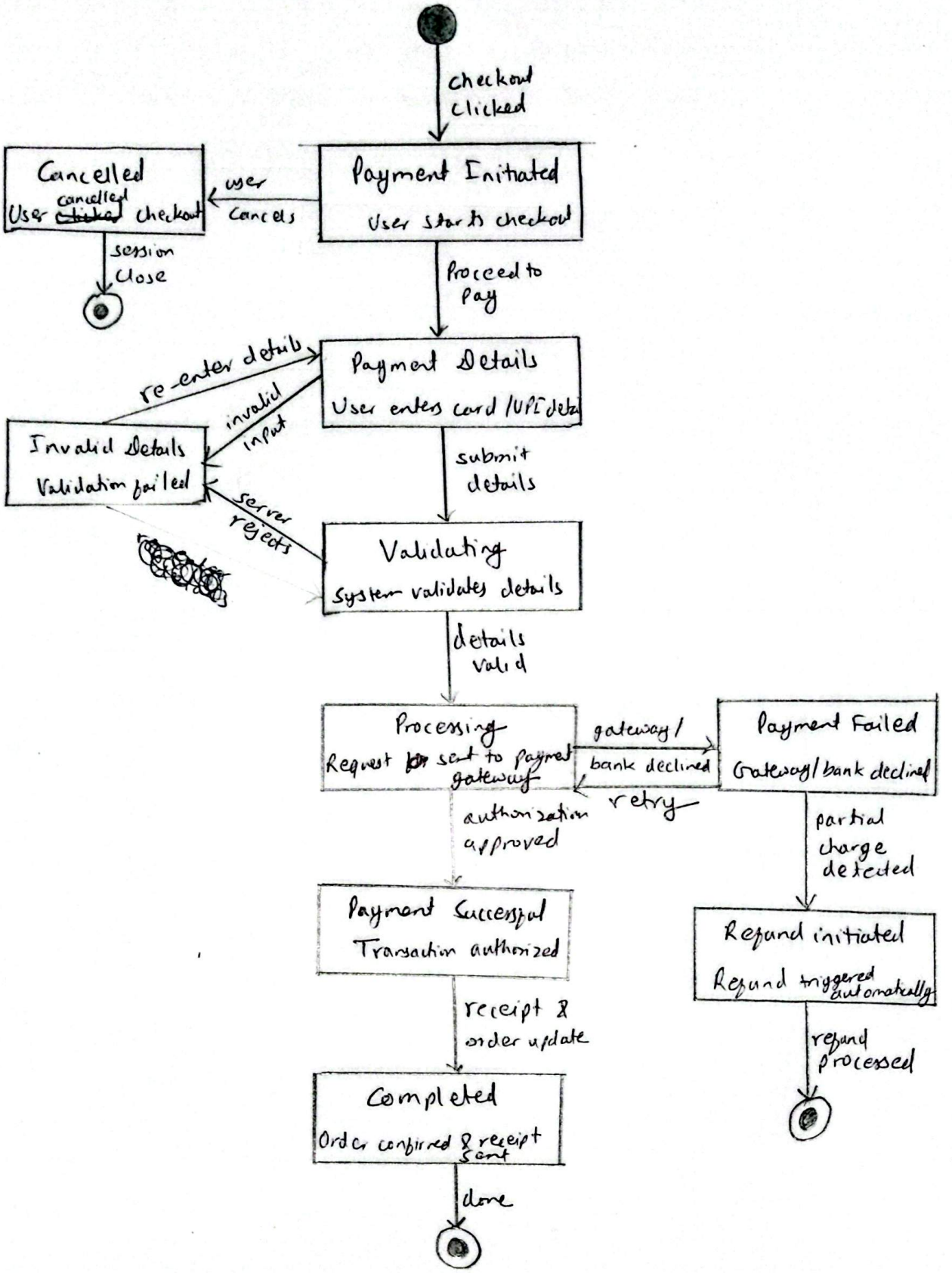


Fig: Activity diagram for Online Payment.

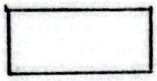
8) Use Case Diagram:

A use case diagram is a UML behavioral diagram used to capture a system's functional requirements - specifically, how different users (actors) interact with that system.

Main components of a Use Case Diagram:

1)  → Actor: represents a user or an outside entity that interacts with the system.

2)  → Usecase: represents a single function or service the system provides.

3)  → ~~Rectangle~~ System Boundary: marks the limits of what falls inside the system being modelled.

4) Relationships:

- Arrows  : Connects an actor to the use cases they interact with.

- <<include>> : One use case will always involve another as part of its own execution

- <<extend>> : shows optional behavior that may be added onto a use case under the certain conditions.

Q 1: Design the usecase diagram for ATM system.

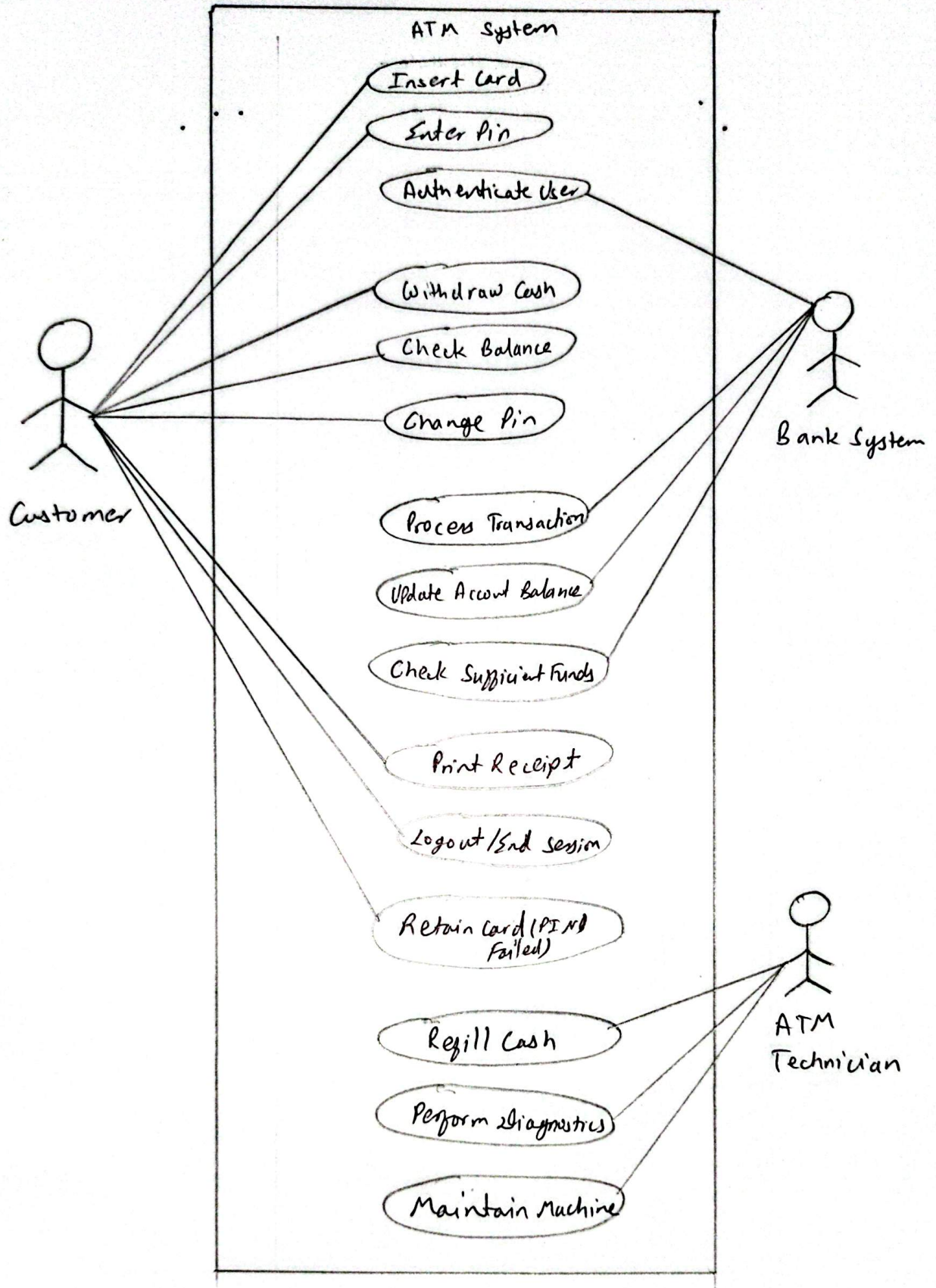
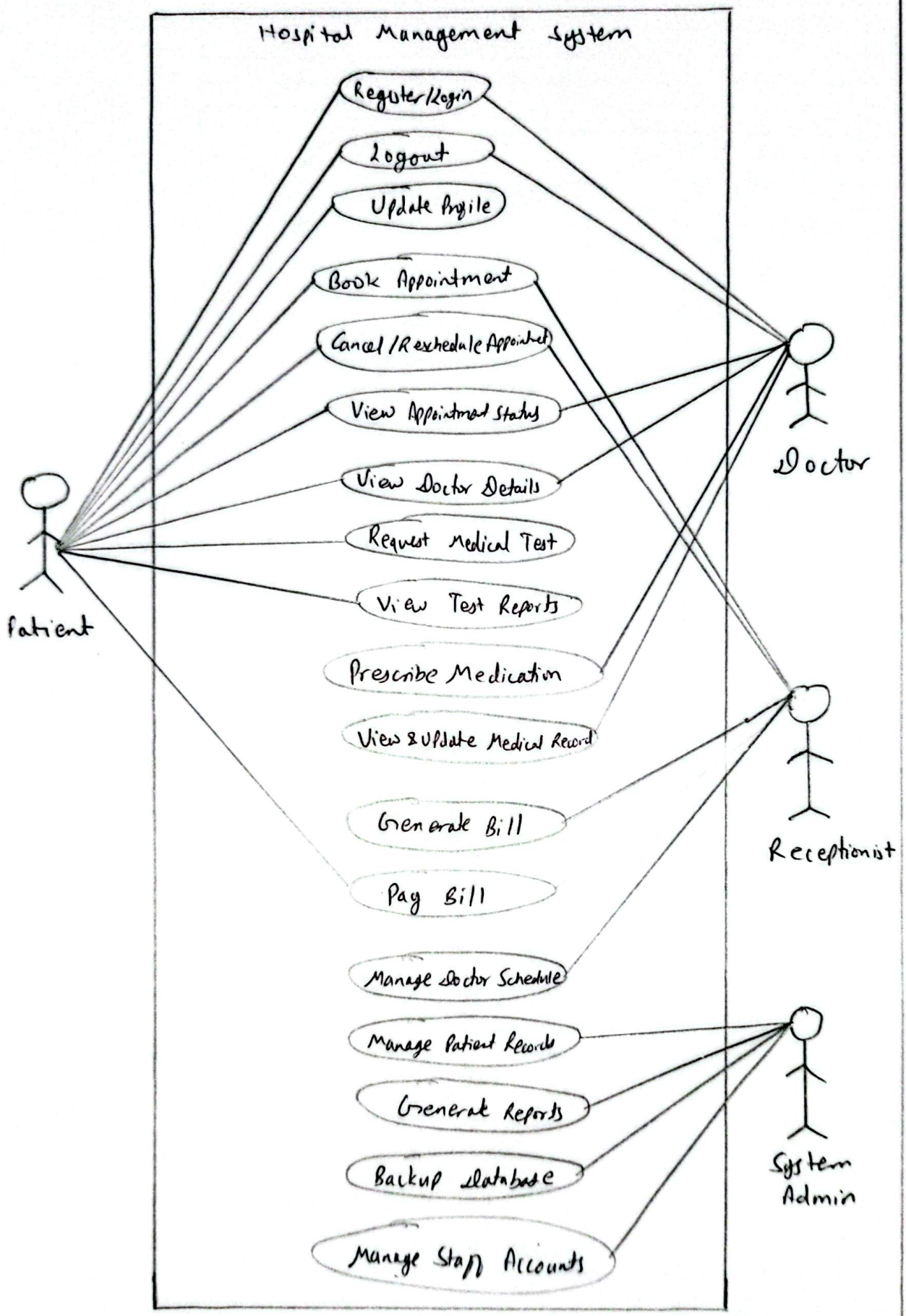


Fig: Use case diagram for ATM Machine

Q:2 Design use case diagram for Hospital Management System



Q.3: Design use case diagram for Student Attendance Management System:

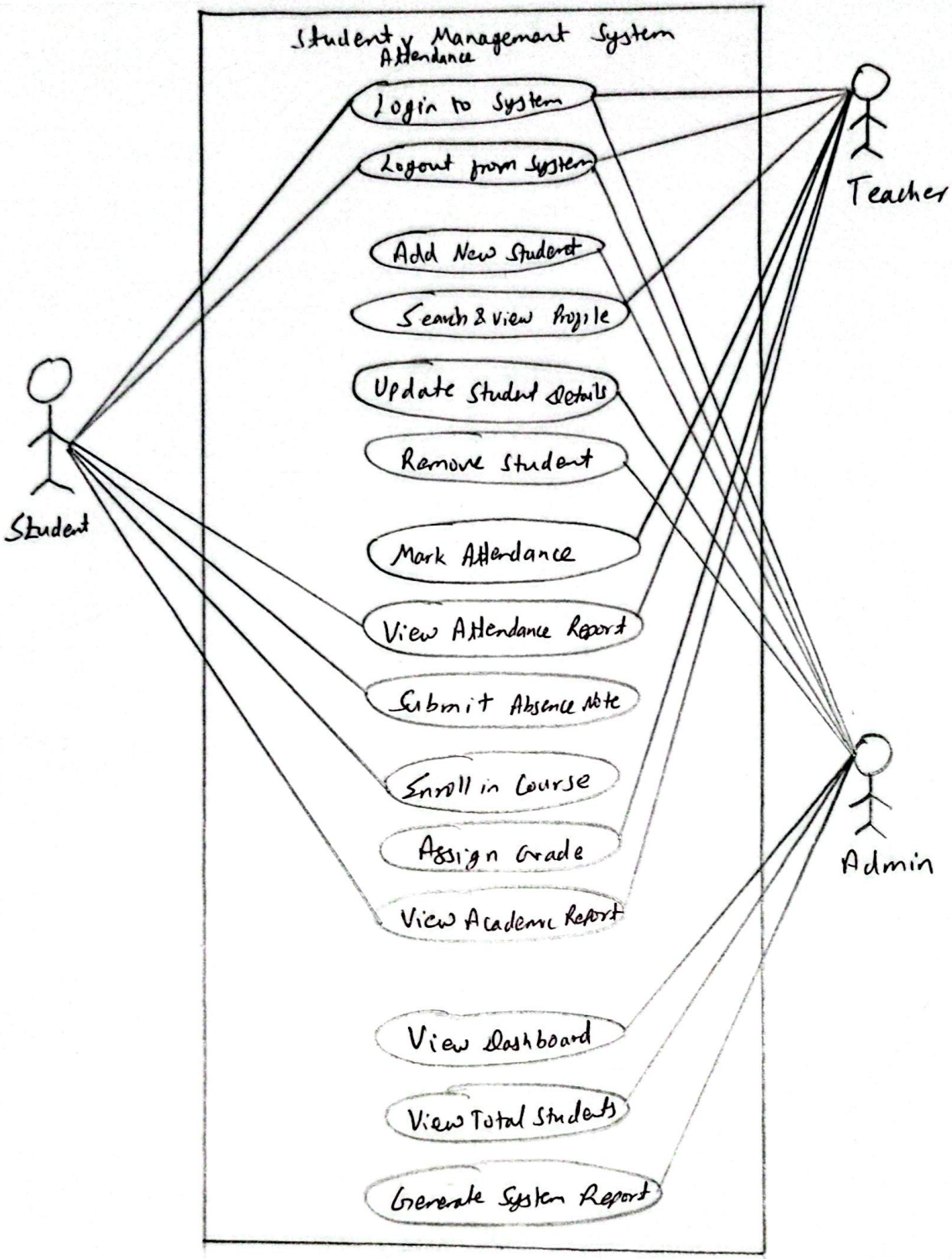
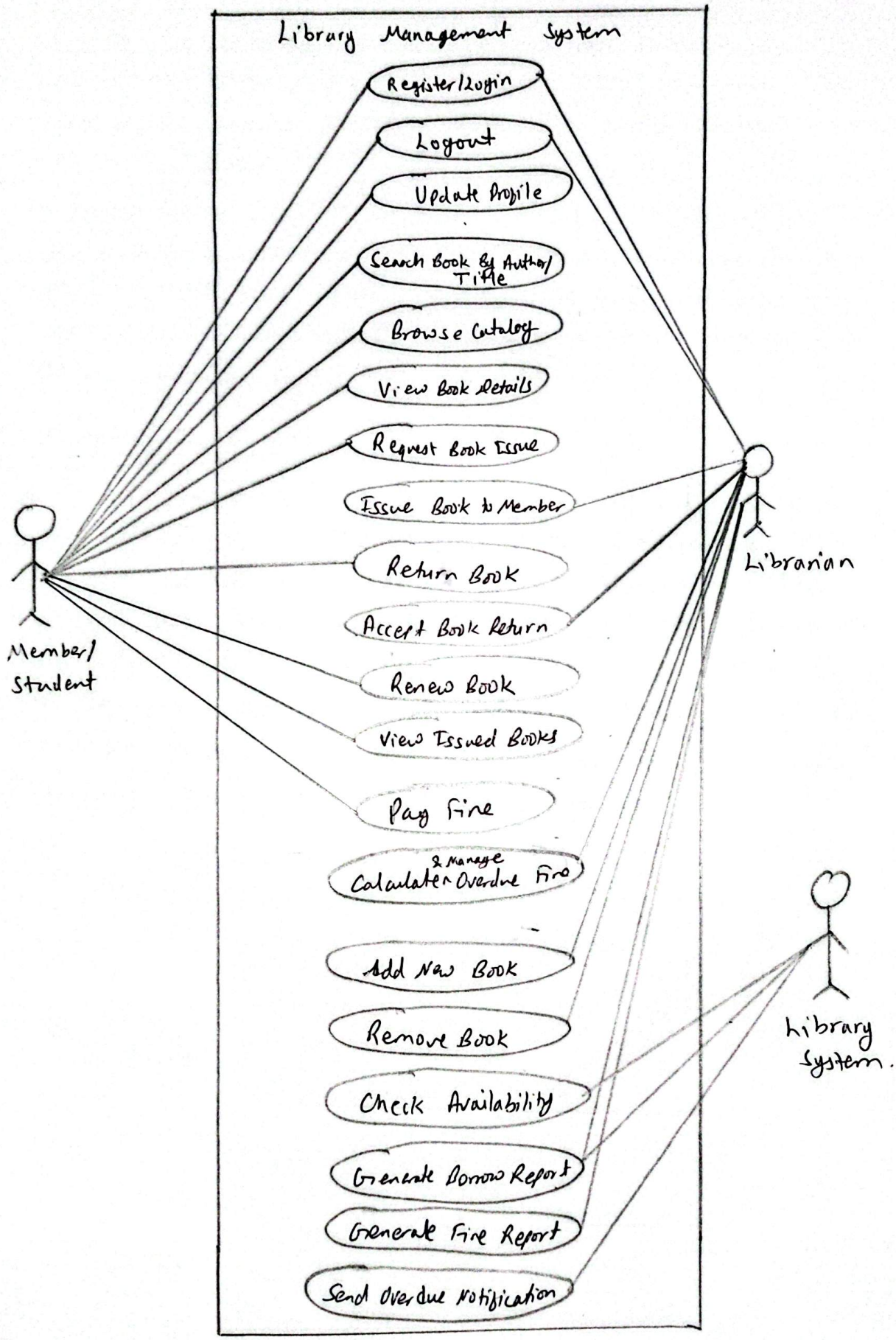


Fig: Use Case Diagram for Student Attendance System

Q.4: Design use case diagram for Library Management System



LAB-3

SCHEDULE

∞ Scheduling is the process of planning and arranging a project's activities in relation to time and the resources that are available. It involves dividing the overall project into smaller, well-defined tasks, estimating how long each task will take, and then fitting those tasks into a project timeline. The goal is to arrange the work in a logical order so that everything is completed within the agreed ~~time~~ deadline. Several scheduling techniques are commonly used in system analysis and design, including Gantt chart (which helps assign responsibilities and visualize timing), the Critical Path Method (CPM), and a PERT (Program Evaluation Review Technique).

* Gantt Chart:

A Gantt chart is a graphical scheduling tool that displays a project's tasks or activities against timeline. Tasks are listed along the vertical axis, while time intervals run along the horizontal axis; and horizontal bars are drawn to indicate when each tasks starts, how long it lasts, and when it finishes. This layout makes it easy to see the sequence in which activities happen, spot ~~tasks~~ tasks that overlap, and keep track of how the project is progressing.

A 1) Prepare a Gantt Chart of a project Schedule Covering all SDLC phases.

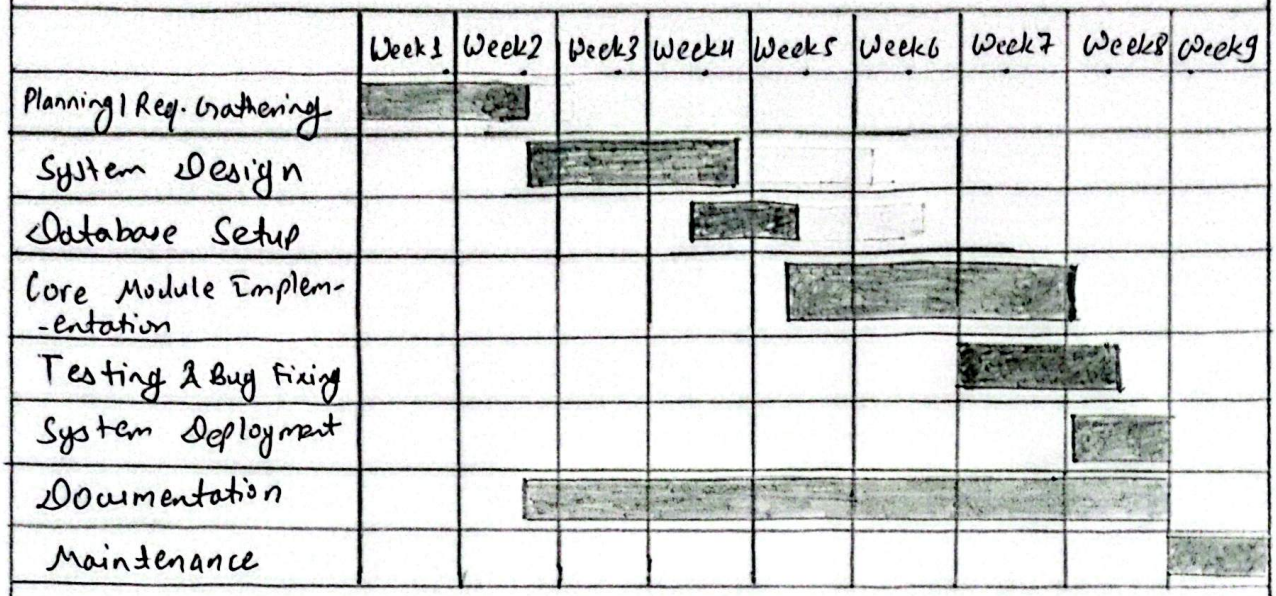


Fig: Gantt Chart